

STUDY GUIDE · KUBERNETES · FROM ZERO TO PRACTICAL

Kubernetes Practical Course

Kubernetes From Zero to Practical

Every one of the 29 lessons, level by level: what it is, why it exists, its diagram, how it works, its common mistakes and its key takeaways. Then the 14 troubleshooting problems in full, the challenges, a glossary and 22 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-kubernetes-practical-course>

Contents

[How to use this study guide](#)

[The method and the big picture](#)

[Level 1 · Kubernetes fundamentals](#)

[Level 2 · kubectl](#)

[Level 3 · Pods](#)

[Level 4 · Deployments](#)

[Level 5 · Services](#)

[Level 6 · Configuration](#)

[Level 7 · Storage](#)

[Level 8 · Health checks](#)

[Level 9 · Resources and scheduling](#)

[Level 10 · Jobs, CronJobs and DaemonSets](#)

[Level 11 · Stateful applications](#)

[Level 12 · Ingress and networking](#)

[Level 13 · Security and RBAC](#)

[Level 14 · Scaling and autoscaling](#)

[Level 15 · Troubleshooting and Helm](#)

[Level 16 · The capstone](#)

[Lab 12 · Troubleshooting](#)

[Problem 01 · Pod stuck in Pending](#)

[Problem 02 · CrashLoopBackOff](#)

[Problem 03 · ImagePullBackOff](#)

[Problem 04 · Container exits immediately](#)

[Problem 05 · Service cannot reach Pods](#)

[Problem 06 · Readiness probe failing](#)

[Problem 07 · Wrong ConfigMap](#)

[Problem 08 · Wrong Secret](#)

[Problem 09 · PVC stuck in Pending](#)

[Problem 10 · Ingress not working](#)

[Problem 11 · Deployment rollout stuck](#)

[Problem 12 · Pod OOMKilled](#)

[Problem 13 · DNS resolution failure](#)

[Problem 14 · NetworkPolicy blocks required traffic](#)

[Challenges](#)

[Glossary](#)

[Kubernetes interview questions](#)

The full lessons, with every manifest, every command and its real, tested output, the labs, the challenges and the capstone, are in the repository.

How to use this study guide

This guide is the revision companion to the course. Every lesson is condensed to what it is, why it exists, its architecture diagram and its key takeaways, level by level. The full lessons, with every manifest, every command, its real output, the labs, the challenges and their solutions, live in the repository: each summary links to its lesson.

How to study

1. **Do the lesson first, read the summary afterwards.** Kubernetes is learned with your hands: apply every manifest, run every command, break the lesson the way its *Break It* section does, and read the real events before you read the explanation.
2. **Events first.** When something is wrong, `kubectl describe` and `kubectl get events` almost always name the cause: `FailedScheduling`, `ImagePullBackOff`, `Readiness probe failed`, `OOMKilled`. The course teaches you to trust them.
3. **Follow the chain.** Ingress → Service → endpoints → Pods → events → logs → configuration → dependencies → policies. Most problems are one wrong link in that chain.
4. **Do the lab of each level**, and the challenges, before reading their solutions.
5. **Finish with the troubleshooting lab and the capstone.** The capstone breaks a complete application six ways.

A study plan

Week	Levels	Goal
1	1–4	a cluster, kubectl, Pods, labels, ReplicaSets, Deployments, rolling updates
2	5–8	Services and DNS, ConfigMaps and Secrets, storage, health checks
3	9–13	resources, scheduling, Jobs, DaemonSets, StatefulSets, Ingress, NetworkPolicies, RBAC
4	14–16	scaling and the HPA, troubleshooting, Helm, the capstone

One hour a day is enough: a lesson takes 20–45 minutes, plus its challenge.

The method and the big picture

The learning method

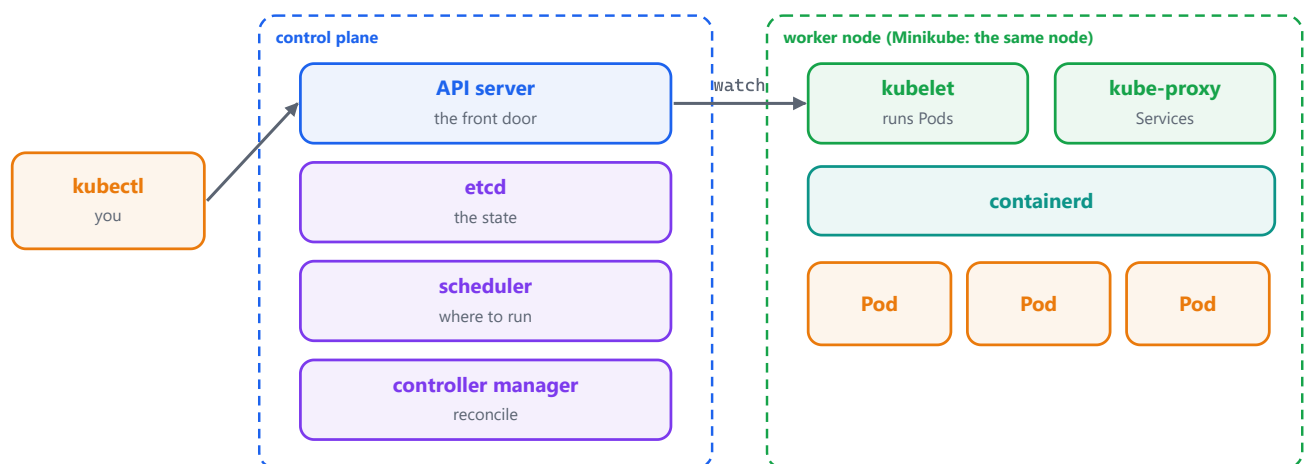
Every lesson walks the same path:

Theory → Visual explanation → YAML → kubectl command → Deploy → Inspect → Experiment → Break it → Troubleshoot → Fix it

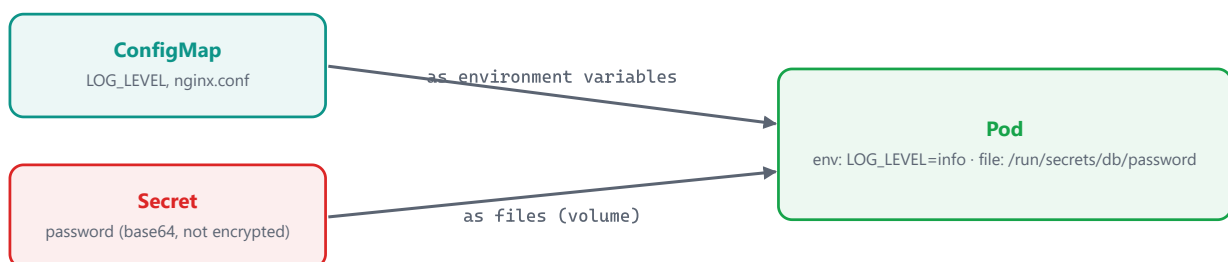
and has the same sections: What is it? · Why do we need it? · How does it work? · Architecture · YAML · Hands-On Lab · Expected Result · Inspect · Experiment · Break It · Troubleshoot It · Common Mistakes · Best Practices · Challenge · Solution · Key Takeaways · Cleanup.

Diagrams

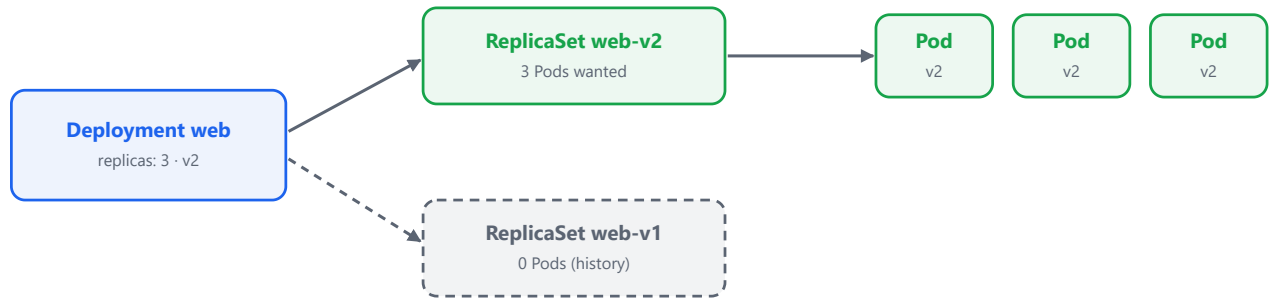
Kubernetes architecture



ConfigMaps and Secrets: configuration outside the image



Deployment → ReplicaSet → Pods

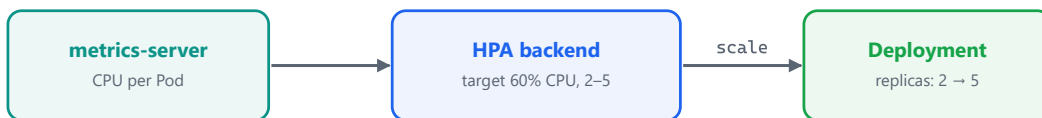


delete a Pod → the ReplicaSet creates one · new version → new ReplicaSet · rollback → old one

Helm: one chart, many environments

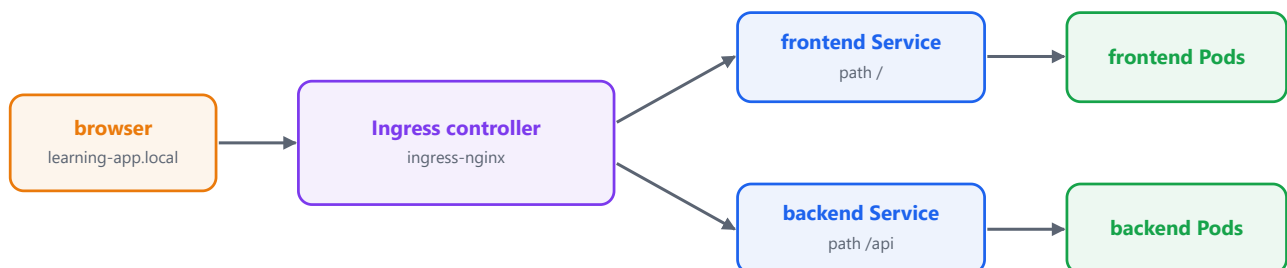


Horizontal Pod Autoscaler



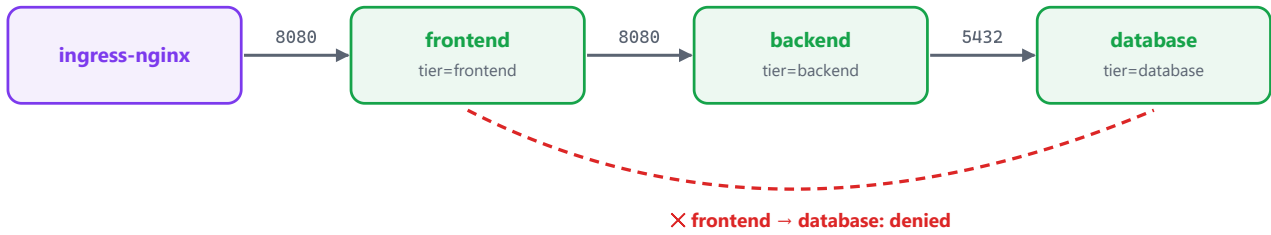
$\text{desired} = \text{ceil}(\text{current} \times \text{currentCPU} / \text{targetCPU})$ · scale-down waits a stabilization window

Ingress: HTTP routing to Services

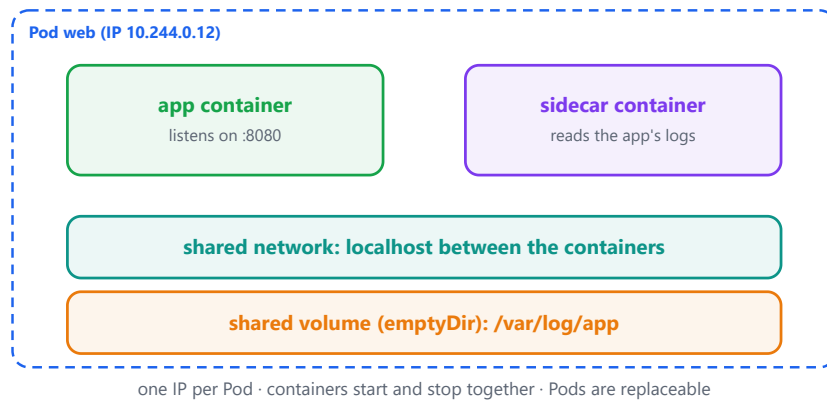


NetworkPolicies: only the allowed paths

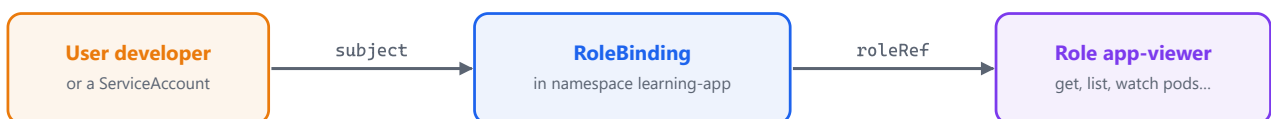
default deny, then explicit allows (enforced by Calico)



A Pod: containers sharing network and storage

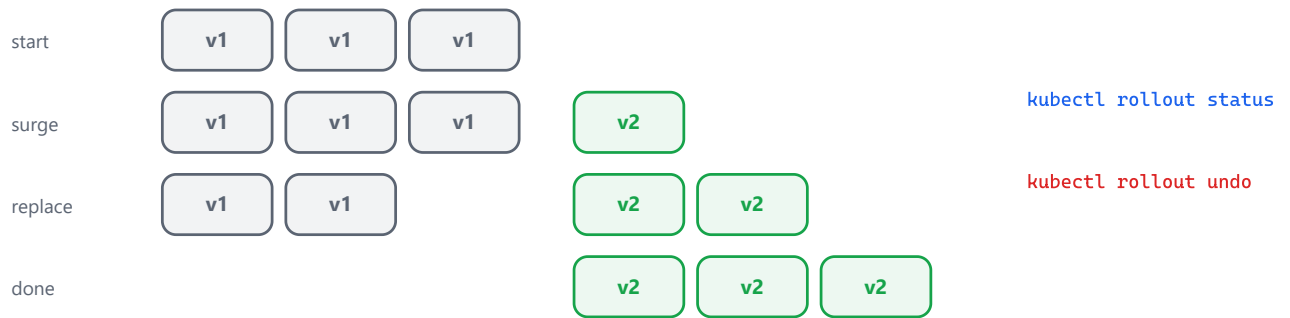


RBAC: who may do what

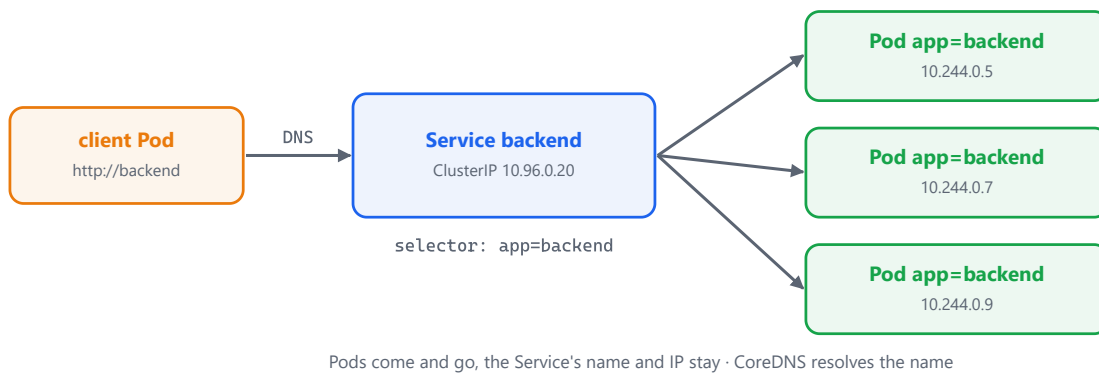


`kubectl auth can-i delete pods --as developer → no`

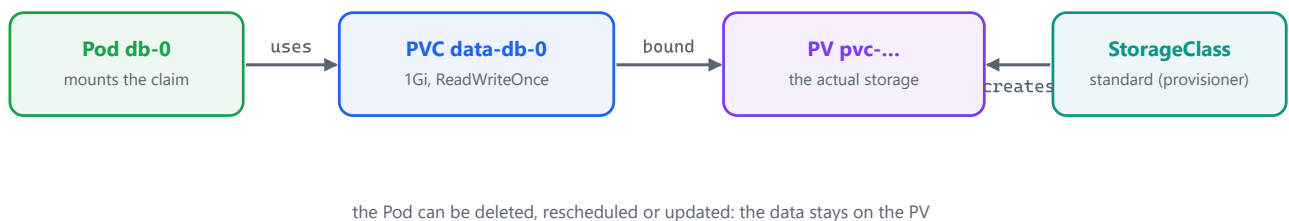
Rolling update and rollback



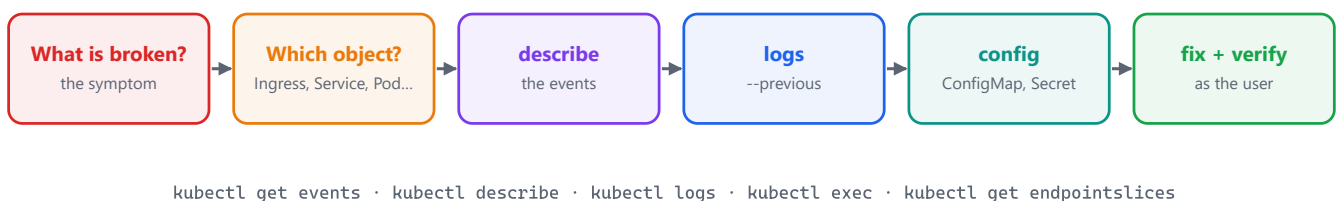
Service: one stable name for changing Pods



Storage: PV ← PVC ← Pod



The troubleshooting path



Level 1 · Kubernetes fundamentals

01 · What is Kubernetes?

Kubernetes is a system that **runs and looks after containerized applications for you**, on one or many machines. You tell it *what you want* ("three copies of my web server, reachable on one address, always running") and it works continuously to make reality match: it starts the containers, restarts them when they crash, replaces them when a machine dies, spreads traffic across them and updates them without downtime.

An analogy: Docker is a skilled cook who can prepare any dish (container) you ask for. Kubernetes is the **restaurant manager**: it decides which cook prepares what, notices when a cook goes home sick and hands the work to another, opens more stations when the restaurant is busy, and keeps the menu (your desired state) written on the wall.

WHY

With one container on one machine, Docker is enough. Applications rarely stay that small:

```
1 container      → docker run is enough
many containers  → which ones are running? which crashed?
multiple machines → which machine runs what? what if one dies?
scaling          → start 5 more copies, spread the traffic
self-healing     → restart what crashed, without a human awake at 3 a.m.
rolling deployments → update to v2 with no downtime, go back to v1 if v2 is broken
                 → Kubernetes
```

Doing all of that by hand, with scripts around `docker run`, is exactly the work Kubernetes automates.

HOW IT WORKS

You never start containers yourself. You **describe the desired state** in YAML files and send them to Kubernetes:

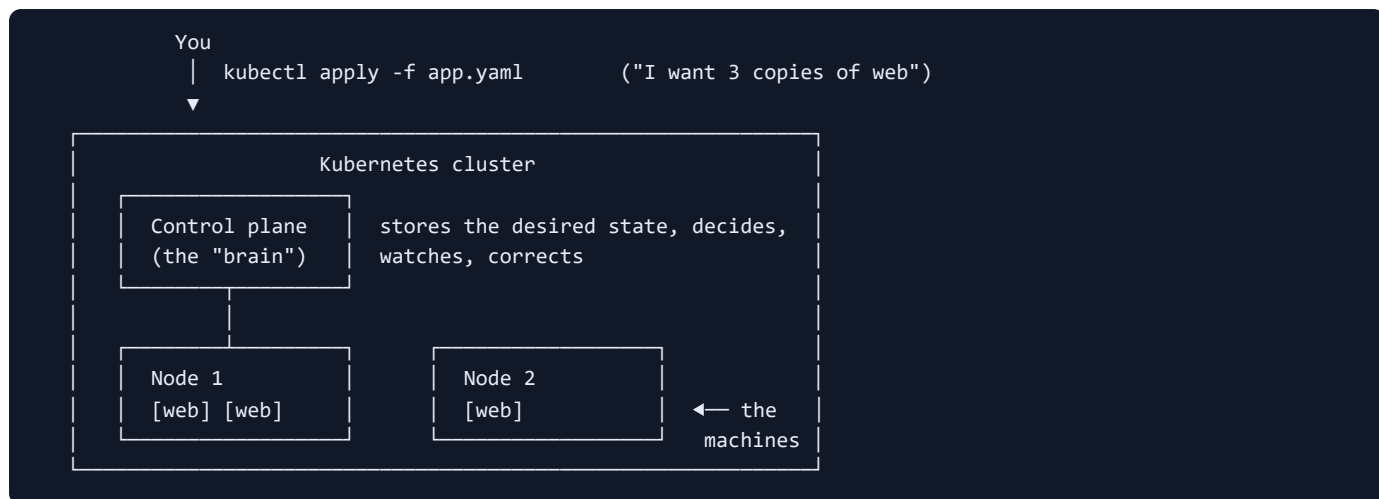
```
you          Kubernetes          machines (nodes)
"3 copies of web:1.0" → stores the desired state → starts 3 containers
                  compares it with reality, forever
                  "only 2 running, I want 3" ← one crashes
                  starts a replacement          → 3 running again
```

This loop (desired state → observe → correct) is called **reconciliation**, and it is the core idea of Kubernetes. Everything in this course is a variation of it.

	Docker	Kubernetes
What it does	runs containers	manages containerized applications
Unit	a container	a Pod (one or more containers, lesson 06)
Scope	one machine	a cluster of machines
You say	"run this container now"	"this is how it should be; keep it that way"
When a container dies	it stays dead (unless a restart policy on that machine)	it is replaced, on any healthy machine
Scaling	start more containers yourself, each on its own port	<code>replicas: 5</code>
Updates	stop old, start new (downtime)	rolling update, rollback

Kubernetes does not replace Docker images: it runs the **same images** you build with Docker.

ARCHITECTURE



Lesson 02 opens the brain and the nodes, one component at a time.

COMMON MISTAKES

Mistake	What happens	Instead
Thinking Kubernetes replaces Docker images	confusion about where images come from	Kubernetes runs the same images you build with Docker
Thinking you need Kubernetes for one container	a lot of complexity for nothing	one container on one machine: Docker (or Compose) is enough
Starting containers by hand on a cluster	Kubernetes does not know them, will not heal them	describe the desired state; let Kubernetes start them

KEY TAKEAWAYS

- Docker runs containers; Kubernetes manages containerized applications across machines.
- You describe the desired state; Kubernetes reconciles reality with it, continuously.
- Self-healing, scaling, one address for many copies and zero-downtime updates are what you get.
- Real-world use: almost every company that runs many services in containers runs them on Kubernetes, in the cloud (EKS, AKS, GKE) or in its own data center; the concepts in this course are the same everywhere.

Full lesson with every command and its real output: [01-kubernetes-introduction](#)

02 · Kubernetes architecture

A Kubernetes **cluster** is a group of machines (**nodes**) with one "brain", the **control plane**. The control plane stores what you want, decides where things run and corrects anything that drifts. The **worker nodes** run your containers. Every component is a separate program with one job, and they all talk through one door: the **API server**.

Back to the restaurant from lesson 01: the control plane is the office (the order book, the person who assigns work, the supervisors who check), and the nodes are the kitchens where the cooking happens.

WHY

When something goes wrong in Kubernetes, the symptom tells you which component to look at, but only if you know what each one does. A Pod that stays `Pending` forever, a Pod that is scheduled but never starts, a cluster where nothing changes any more: each points at a different component. This lesson builds that map.

HOW IT WORKS

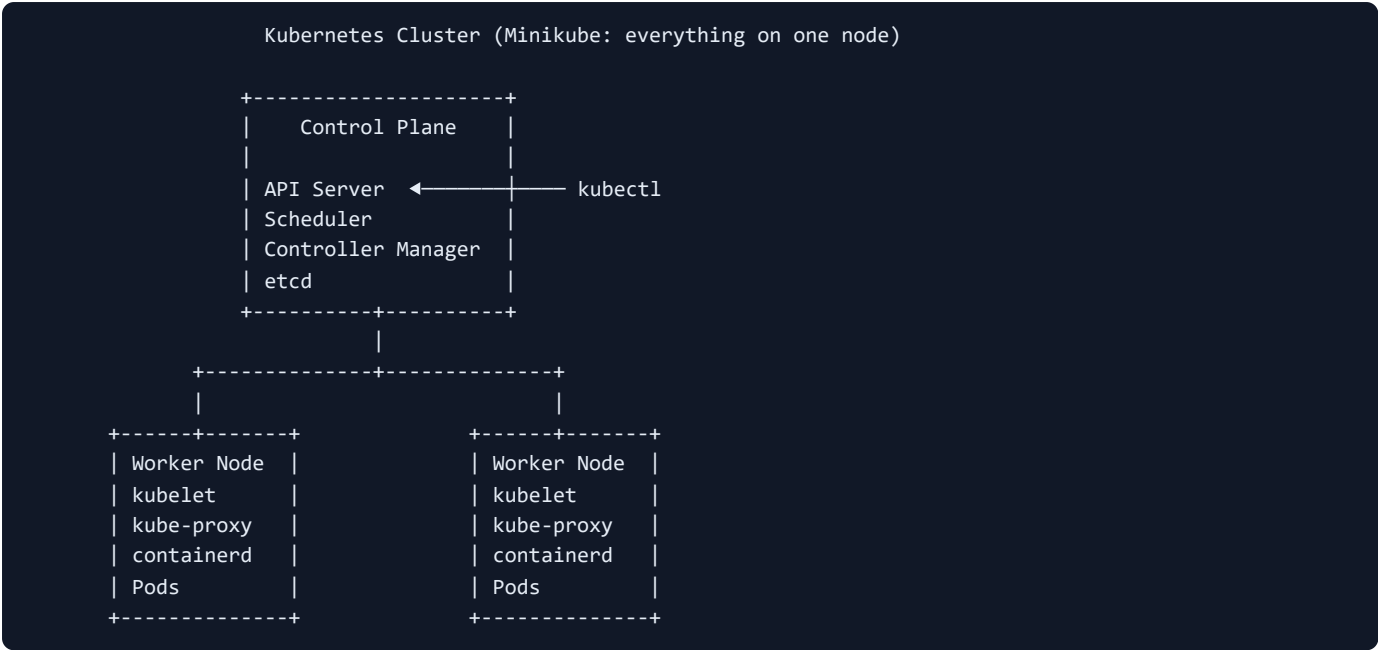
Component	Where	Its one job (beginner version)
API server (<code>kube-apiserver</code>)	control plane	the front door: every command (<code>kubectl</code>), every component, talks only to it
etcd	control plane	the database: stores the desired and current state of every object
Scheduler (<code>kube-scheduler</code>)	control plane	picks a node for every new Pod that has none
Controller manager (<code>kube-controller-manager</code>)	control plane	runs the reconciliation loops: "3 copies wanted, 2 running → create 1"
kubelet	every node	the node agent: starts the Pod's containers, reports their status
kube-proxy	every node	makes Service addresses (lesson 10) work on the node
Container runtime (containerd)	every node	actually runs containers (what Docker did on your laptop)

What happens when you create a Pod:

```
kubectl run web → API server → etcd          (stored: "Pod web, no node yet")
                    ▲
scheduler ————— sees a Pod without a node, picks one, writes "node = minikube"
kubelet on minikube — sees a Pod assigned to it, asks containerd to start the container,
                      reports "Running" back to the API server
```

Nobody gives orders directly: every component **watches** the API server for objects it cares about and acts. That is why one missing component stops exactly one step of the chain.

ARCHITECTURE



Minikube runs the control plane and the worker on **one** node (named `minikube`); production clusters keep them on separate machines. The components are the same.

COMMON MISTAKES

Mistake	Symptom	Fix
Expecting <code>kubectl</code> to talk to the nodes	confusion when the API server is down: every command fails	everything goes through the API server: check it first (<code>kubectl get --raw /livez</code>)
Deleting a <code>kube-system</code> Pod to "restart" a static Pod	the Pod comes back unchanged	static Pods are owned by the kubelet: change the file in <code>/etc/kubernetes/manifests</code>
Reading <code>Pending</code> as "image is downloading"	wrong investigation	<code>Pending</code> with no node = scheduling; with a node = the kubelet (image pull, volumes)

KEY TAKEAWAYS

- Control plane: API server (front door), etcd (state), scheduler (where), controller manager (reconcile).
- Every node: kubelet (start containers), kube-proxy (Service networking), container runtime (containerd).
- Components watch the API server and act; a missing component stops one step: `Pending` without node → scheduler.
- Real-world use: reading which component failed from a symptom is the first skill of on-call Kubernetes work.

Full lesson with every command and its real output: [02-architecture](#)

03 · Installing Minikube

Minikube creates a real Kubernetes cluster on your computer: one node, running inside a Docker container. It is the same Kubernetes as in production (the same API server, scheduler, kubelet...), just small. **kubectl** is the command-line client you use to talk to any Kubernetes cluster, including this one.

WHY

You learn Kubernetes by using it, and you need a cluster you can break without consequences, without a cloud account and without paying anything. Minikube is that cluster; every lesson of this course runs on it.

⚠ **Do not run the commands of this course against a production or shared cluster.** They delete things, stop components and break them on purpose. Always check which cluster `kubectl` talks to: `kubectl config current-context` must print `minikube`.

HOW IT WORKS

```
your computer
├─ Docker (Docker Desktop on Windows/macOS, Docker Engine on Linux)
│   └─ container "minikube" ← the Kubernetes node, a small Linux system
│       ├── containerd ← the container runtime inside the node
│       ├── kubelet, control plane ← Kubernetes itself
│       └─ your Pods ← containers started by Kubernetes
├─ minikube ← creates, starts, stops and deletes that node
└─ kubectl ← talks to the API server inside it; minikube writes the address into ~/.kube/config
```

Alternatives exist (kind, k3d, Docker Desktop's Kubernetes); they all give you a local cluster. This course uses Minikube everywhere, with one set of settings, so every output you see matches yours.

ARCHITECTURE

```
kubectl → ~/.kube/config → https://127.0.0.1:PORT → API server |
              (context "minikube")                               | inside the
minikube → Docker → container "minikube" (the node) ————— | "minikube" container
```

COMMON MISTAKES

Problem	Symptom	Fix
Docker is not running	<code>PROVIDER_DOCKER_NOT_RUNNING</code> or Cannot connect to the Docker daemon	start Docker Desktop / <code>sudo systemctl start docker</code> , then <code>minikube start</code> again
Docker driver problems	<code>minikube start</code> hangs or fails at "Creating docker container"	<code>minikube delete</code> , check <code>docker info</code> works, start again; on Linux your user must be in the <code>docker</code> group
kubectl cannot connect	connection refused / localhost:8080	<code>minikube status</code> ; if running, <code>minikube update-context</code> rewrites the kubeconfig entry
The cluster is stopped	every kubectl command fails	<code>minikube start</code> with the same flags
Insufficient resources	<code>RSRC_OVER_ALLOC_MEM</code> , or Pods Pending with Insufficient memory later	lower <code>--memory</code> / <code>--cpus</code> , or give Docker Desktop more in its settings
kubectl points to another cluster	commands hit a work cluster	<code>kubectl config current-context</code> must say <code>minikube</code> ; <code>kubectl config use-context minikube</code>

KEY TAKEAWAYS

- Minikube = a real one-node Kubernetes cluster inside a Docker container; kubectl = the client for any cluster.
- The course cluster: `minikube start --driver=docker --cni=calico --cpus=2 --memory=4g`.
- `minikube status` and `kubectl config current-context` are the first checks when kubectl cannot connect.
- Real-world use: every DevOps engineer keeps a local cluster to try manifests, upgrades and failures before they reach a shared cluster.

Full lesson with every command and its real output: [03-installation](#)

Level 2 · kubectl

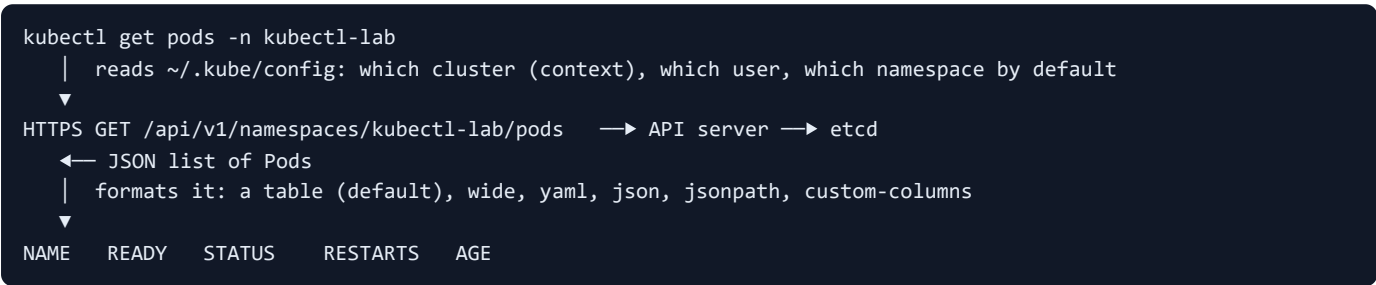
04 · kubectl fundamentals

`kubectl` ("kube control") is the command-line client of Kubernetes. Every `kubectl` command becomes a request to the API server: *list these objects, create this one, show me its logs*. Learning `kubectl` is learning to ask the cluster questions and to read its answers.

WHY

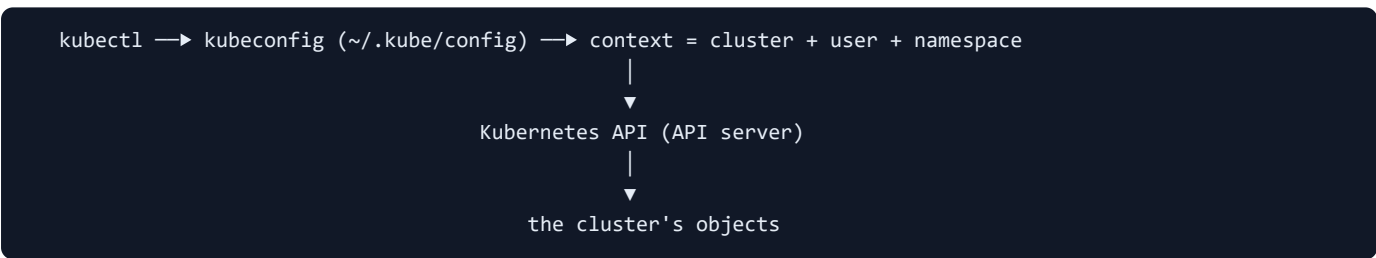
Kubernetes has no screen of its own; the API is the only way in. `kubectl` is the tool every Kubernetes engineer uses daily, on every cluster, cloud or local. The same twenty commands cover almost all of the work: looking (get, describe, logs, events), changing (apply, delete, edit, scale) and debugging (exec, port-forward).

HOW IT WORKS



The pattern is always `kubectl VERB [TYPE] [NAME] [flags]`: `kubectl get pod web -n kubectl-lab -o yaml`.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Forgetting <code>-n</code>	<code>NotFound</code> , No resources found in default namespace	<code>-n NS</code> , <code>-A</code> to search everywhere, or set the context's namespace
<code>kubectl create -f</code> twice	<code>AlreadyExists</code>	<code>kubectl apply -f</code> creates or updates
Wrong context	commands hit another cluster	<code>kubectl config current-context</code> before every delete
Typo in a field	unknown field "imgae"	<code>kubectl explain</code> the path; <code>--dry-run=server</code> checks before applying
Parsing tables in scripts	breaks when columns change	<code>-o jsonpath</code> , <code>-o custom-columns</code> , <code>-o json</code>

KEY TAKEAWAYS

- `kubectl VERB TYPE NAME -n NS -o FORMAT`: get, describe, logs, exec, apply, delete, explain, port-forward, config.
- `apply -f` is declarative ("make it so"); `create` is imperative ("do this once").
- `describe` + `get events` explain problems; `-o wide/yaml/custom-columns/jsonpath` show exactly what you need.

- Real-world use: kubectl is the daily tool of every Kubernetes engineer, for deploying, inspecting and debugging.

Full lesson with every command and its real output: [04-kubectl](#)

05 · Namespaces

A **namespace** is a named folder inside the cluster. Most objects (Pods, Deployments, Services, ConfigMaps...) live in exactly one namespace, and their names only have to be unique inside it. One cluster can hold `dev`, `test` and `prod` side by side, each with its own `web` Pod.

Analogy: one office building (the cluster), many companies on different floors (namespaces). Two companies can both have a "Reception" room; everybody knows which one you mean by the floor.

WHY

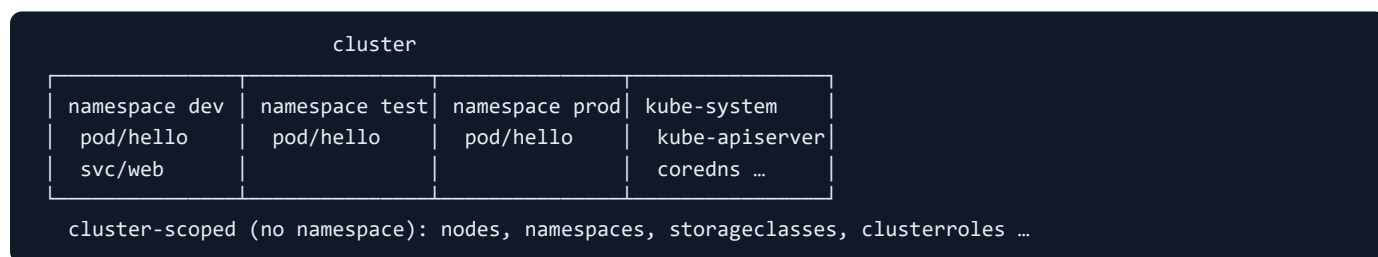
- **Organization:** a team, an application or an environment gets its own space; `kubectl get pods -n dev` shows only what belongs there.
- **Isolation of names:** two teams can both call something `web`.
- **Resource management and access control:** quotas (how much CPU and memory a namespace may use) and permissions (RBAC, lesson 24) are set per namespace.
- **Clean-up:** deleting a namespace deletes everything inside it, in one command.

HOW IT WORKS

Every cluster starts with a few namespaces: `default` (where things go when you name none), `kube-system` (the cluster's own components, lesson 02), `kube-public` and `kube-node-lease`. A **namespaced** object belongs to one namespace; a **cluster-scoped** object (a Node, a Namespace itself, a StorageClass, a ClusterRole) belongs to the whole cluster. `kubectl` uses the namespace you give with `-n`, or the default namespace of your context.

Namespaces separate *names and permissions*, not the network: by default a Pod in `dev` can reach a Pod in `prod`. NetworkPolicies (lesson 22) add network isolation.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Forgetting <code>-n</code>	No resources found in default namespace	<code>-n NS</code> , or <code>-A</code> to search all namespaces
Applying objects before their namespace	namespaces "x" not found	create the namespace first
<code>kubectl delete namespace</code> to "clean a bit"	everything inside is deleted, irreversibly	delete individual objects; delete a namespace only when all of it should go
Expecting namespaces to block network traffic	Pods in <code>dev</code> reach Pods in <code>prod</code>	NetworkPolicies (lesson 22)
Putting your own objects in <code>kube-system</code>	mixed with cluster components, hard to clean	one namespace per application or environment

KEY TAKEAWAYS

- A namespace is a named space for objects; names are unique inside it, not across the cluster.

- `-n NS` chooses the namespace, `-A` searches all; without them kubectl uses the context's default (`default`).
- Deleting a namespace deletes everything in it.
- Real-world use: separating applications, teams and environments, and attaching quotas and permissions to them.

Full lesson with every command and its real output: [05-namespaces](#)

Level 3 · Pods

06 · Pods

A **Pod** is the smallest thing Kubernetes runs: one or more containers that always run **together, on the same node**, sharing one network address and, optionally, storage. Kubernetes never runs a container on its own; it runs Pods, and a Pod runs containers.

Analogy: a container is a person; a Pod is a small apartment. Most apartments have one resident. Sometimes two people live together because they work closely: they share the address (network), the kitchen (volumes) and they move in and out together.

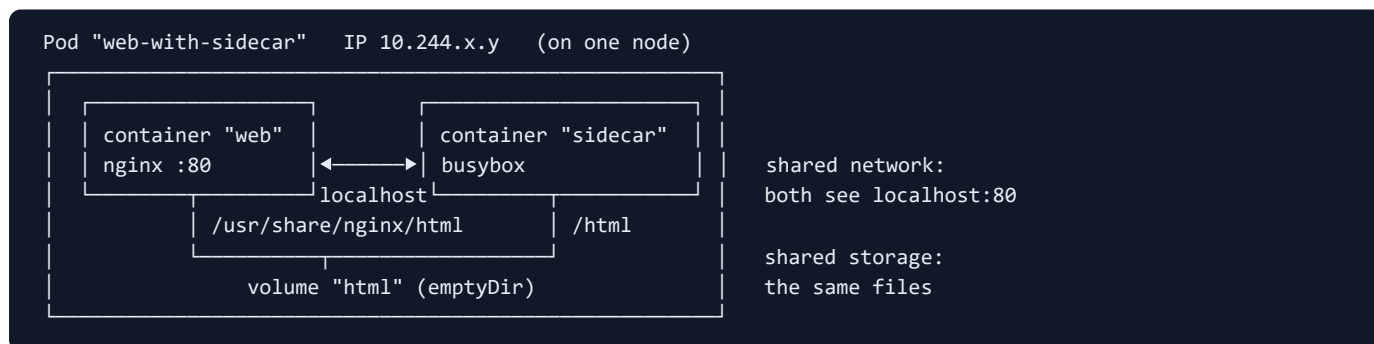
WHY

Some processes belong together: a web server and a helper that refreshes its files, an application and an agent that ships its logs. They must run on the same machine, talk over `localhost` and share files. The Pod is the unit that guarantees this. It is also the unit Kubernetes schedules, gives an IP address, restarts and reports on.

HOW IT WORKS

- **Pod IP:** every Pod gets its own IP address inside the cluster; all its containers share it (and its ports).
- **Shared network namespace:** containers in a Pod reach each other on `localhost`; two containers in one Pod cannot both listen on the same port.
- **Shared volumes:** a volume declared in the Pod can be mounted into several of its containers.
- **Lifecycle (phase):** `Pending` (accepted, not running yet: scheduling, image download) → `Running` (at least one container running) → `Succeeded` (all containers exited with 0) or `Failed`. The kubelet restarts crashed containers according to `restartPolicy` (`Always` by default).
- **Pods are mortal:** a Pod is never moved or resurrected. If it is deleted, or its node dies, it is gone; something else (a ReplicaSet, lesson 08) must create a new one, with a new name and a new IP. That is why you rarely create bare Pods in real life.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
A container whose command ends	<code>Completed</code> , then <code>CrashLoopBackOff</code>	the main process must keep running (a server, not a script)
Two containers on the same port in one Pod	the second one fails: <code>address already in use</code>	different ports; they share one network
<code>logs / exec</code> without <code>-c</code> on a multi-container Pod	the wrong container	<code>-c NAME</code>
Editing a Pod's image or command in place	<code>Forbidden: pod updates may not change fields</code>	delete and recreate, or use a Deployment (lesson 09)
Creating bare Pods for applications	a deleted Pod never comes back	Deployments (lessons 08–09)

KEY TAKEAWAYS

- A Pod is one or more containers on one node, sharing an IP (localhost) and volumes.
- Phases: Pending → Running → Succeeded/Failed; `CrashLoopBackOff` = the container keeps exiting.
- `describe` (events, exit code) and `logs --previous` explain crashes.
- Bare Pods are never recreated; controllers (ReplicaSets, Deployments) manage Pods for you.
- Real-world use: sidecars ship logs, refresh certificates or proxy traffic next to the main application.

Full lesson with every command and its real output: [06-pods](#)

07 · Labels, selectors and annotations

A **label** is a key/value tag on an object: `app: web`, `environment: dev`, `tier: frontend`. A **selector** is a question about labels: "all objects with `app=web` and `environment=prod`". An **annotation** is also a key/value pair, but for information meant for people and tools, never for selecting (a description, a contact, a build number).

Analogy: labels are the coloured stickers on moving boxes ("kitchen", "fragile"); a selector is "bring me every box with a kitchen sticker". Annotations are the notes written on the box: useful to read, never used to sort.

WHY

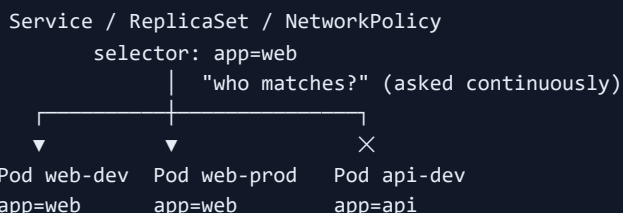
This is the **most important idea for understanding how Kubernetes connects things**. Kubernetes objects do not point at each other by name: a ReplicaSet does not keep a list of "its" Pods, a Service does not list "its" Pods. They hold a **selector**, and whatever currently carries matching labels belongs to them. New Pods with the right labels are picked up automatically; a Pod whose labels change drops out. Lessons 08, 09, 10, 22 and 24 all depend on it.

HOW IT WORKS

Labels	→	Selectors	→	matching objects
<code>app=web</code>		<code>app=web</code>		<code>web-dev</code> , <code>web-prod</code>
<code>environment=dev</code>		<code>environment in (dev)</code>		<code>web-dev</code> , <code>api-dev</code>
<code>tier=frontend</code>		<code>tier!=backend</code>		<code>web-dev</code> , <code>web-prod</code>

- **Equality-based:** `app=web`, `environment!=prod`.
- **Set-based:** `environment in (dev,test)`, `tier notin (backend)`, `app` (has the key), `!app` (has not).
- Several conditions separated by commas must **all** match (AND).
- Keys may have a prefix (`app.kubernetes.io/name`); values are short strings (63 characters, letters, digits, `-`, `_`, `.`).

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Typo or wrong case in a label	selector finds nothing, silently	compare with <code>--show-labels</code>
Using annotations to select	selectors ignore annotations	labels for selecting, annotations for information
Label-based delete without checking	more Pods deleted than intended	<code>--dry-run=client</code> first
Too many ad-hoc labels	nobody knows which ones matter	agree on a few: <code>app</code> , <code>environment</code> , <code>tier</code> , or the <code>app.kubernetes.io/*</code> recommended labels

KEY TAKEAWAYS

- Labels tag objects; selectors find objects by labels; annotations hold information that is never selected.

- Services, ReplicaSets, Deployments and NetworkPolicies find their Pods with selectors, continuously.
- An empty selection is not an error: check labels with `--show-labels` when "nothing happens".
- Real-world use: every routing, scaling, policy and monitoring rule in Kubernetes is written as a label selector.

Full lesson with every command and its real output: [07-labels-selectors](#)

Level 4 · Deployments

08 · ReplicaSets

A **ReplicaSet** keeps a given number of identical Pods running. You say "3 Pods like this"; it counts the Pods that match its selector, and creates or deletes Pods until the count is right, forever.

Analogy: a thermostat. You set 21 °C; it does not heat once, it keeps measuring and corrects whenever the room drifts. A ReplicaSet does that with a number of Pods.

WHY

Lesson 06 showed that a deleted Pod never comes back. One Pod is also one point of failure and has a fixed capacity. Applications need several copies (**replicas**) for availability and load, and they need something that replaces a copy when it dies. That is **self-healing**, and the ReplicaSet is the controller that does it.

HOW IT WORKS

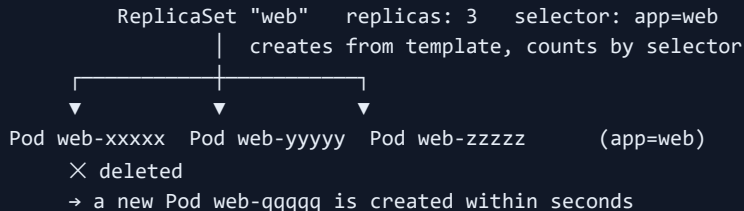
The controller manager (lesson 02) runs a loop for every ReplicaSet:

```
desired = spec.replicas           3
current = Pods matching spec.selector 2 (one was deleted)
desired > current → create Pods from spec.template
desired < current → delete the extra Pods
```

The Pods it creates get the labels of the **template**, and the selector must match those labels; otherwise the ReplicaSet could never count its own Pods, and the API server refuses it.

In practice you rarely create ReplicaSets yourself: a **Deployment** (lesson 09) creates and replaces them for you. Understanding the ReplicaSet is understanding what a Deployment does underneath.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Selector ≠ template labels	selector does not match template labels	the same labels in both places
Two controllers selecting the same labels	Pods deleted and recreated endlessly	unique selector labels per controller
Editing the template of a ReplicaSet	existing Pods keep the old version	a ReplicaSet only creates Pods, never updates them: use a Deployment
Deleting Pods to "fix" them	identical replacements appear	fix the template (or the Deployment)

KEY TAKEAWAYS

- A ReplicaSet keeps `replicas` Pods matching its selector running: self-healing.
- The selector must match the template's labels; Pods are counted by labels, not names.
- A ReplicaSet never updates existing Pods; Deployments manage ReplicaSets to do updates.
- Real-world use: every Deployment you run creates ReplicaSets; you will see them in `kubectl get rs` daily.

Full lesson with every command and its real output: [08-replicasets](#)

09 · Deployments, rolling updates and rollbacks

A **Deployment** is the standard way to run a stateless application on Kubernetes. It manages ReplicaSets (lesson 08), which manage Pods. When you change the Deployment (a new image, a new setting), it creates a **new** ReplicaSet and moves the Pods over to it gradually: a **rolling update**. It keeps the old ReplicaSets, so it can go back: a **rollback**.

Analogy: replacing the tyres of a bus while it keeps driving its route, one wheel at a time, with the old tyres kept in the boot in case the new ones are wrong.

WHY

Lesson 08 ended with a problem: a ReplicaSet never updates its Pods, and replacing them all at once means downtime. Real applications are updated many times a week. A Deployment updates them **without downtime**, stops when something goes wrong, records the **history** of versions and lets you return to a previous one with one command.

HOW IT WORKS

```
Deployment backend      image 1.0.0 → you change it to 2.0.0
├─ ReplicaSet backend-aaa (1.0.0) 3 → 2 → 1 → 0 Pods
└─ ReplicaSet backend-bbb (2.0.0) 0 → 1 → 2 → 3 Pods
```

1. The Deployment controller creates a new ReplicaSet from the changed template.
2. It scales the new one up and the old one down, a few Pods at a time, within two limits: - `maxSurge` : how many Pods **above** replicas may exist during the update (extra capacity); - `maxUnavailable` : how many Pods **below** replicas may be not ready.
3. A new Pod counts only once it is **ready** (its readiness probe passes, lesson 14). A new version that never becomes ready stops the update halfway: the old Pods keep serving.
4. Each template change is a **revision**; old ReplicaSets are kept (scaled to 0) for rollbacks.

ARCHITECTURE

```
Old Pods (v1)  [ ][ ][ ]
Step 1         [ ][ ][ ] [ ] new v2 Pod starting (maxSurge: 1 → 4 Pods)
Step 2         [ ][ ][ ] [ ][ ] v2 ready, one v1 removed (maxUnavailable: 1)
Step 3         [ ][ ][ ] [ ][ ][ ]
Done           [ ][ ][ ] [ ][ ][ ] all v2, old ReplicaSet kept at 0
```

```
Deployment → ReplicaSet (revision 2, current) → Pod Pod Pod
           ↳ ReplicaSet (revision 1, 0 Pods)    (kept for rollback)
```

COMMON MISTAKES

Mistake	Symptom	Fix
A wrong image tag	rollout stuck, a Pod <code>ErrImagePull</code> / <code>ImagePullBackOff</code>	<code>kubectl rollout undo</code> , then fix the tag
No readiness probe	a broken version counts as ready and replaces all old Pods	always define a readiness probe (lesson 14)
Using <code>latest</code>	you cannot tell which version runs, rollbacks do nothing	immutable version tags
<code>kubectl edit</code> in production	the change exists only in the cluster, not in Git	change the file, <code>kubectl apply -f</code>

Mistake	Symptom	Fix
Changing spec.selector	field is immutable	the selector is fixed at creation; create a new Deployment

KEY TAKEAWAYS

- Deployment → ReplicaSets → Pods; a template change creates a new ReplicaSet and a rolling update.
- maxSurge and maxUnavailable control the pace; readiness decides when a new Pod counts.
- rollout status , rollout history , rollout undo watch, explain and revert.
- A broken version stops the rollout halfway while the old Pods keep serving.
- Real-world use: every stateless service (web apps, APIs, workers) is a Deployment; CI/CD pipelines change its image.

Full lesson with every command and its real output: [09-deployments](#)

Level 5 · Services

10 · Services, service discovery and DNS

A **Service** gives a group of Pods **one stable address and name**, and spreads the traffic sent to it over all of them. The Pods behind it are chosen with a label selector (lesson 07). Kubernetes' **DNS** turns the Service's name (`backend-service`) into that address, so applications call each other by name.

Analogy: a company's switchboard number. Employees (Pods) come and go and change desks (IP addresses); customers always dial the same number, and the switchboard connects them to whoever is available.

WHY

Pod IPs change. Every Pod gets a new IP when it is recreated (a crash, a rolling update, scaling), and there are several Pods. A frontend cannot keep a list of backend Pod IPs up to date. It needs one address that always leads to the healthy backend Pods: a Service.

HOW IT WORKS

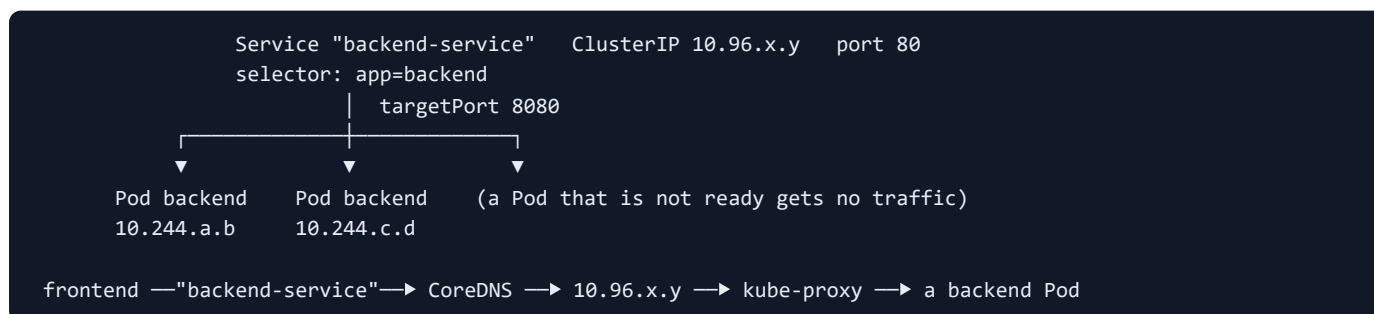
```
frontend Pod: wget http://backend-service
| 1. DNS (CoreDNS): backend-service → 10.96.x.y (the Service's ClusterIP, stable)
| 2. kube-proxy rules on the node: 10.96.x.y:80 → one of the ready Pods, port 8080
▼
backend Pods 10.244.a.b:8080, 10.244.c.d:8080 ← kept up to date in EndpointSlices, from the selector
```

- The Service's **selector** finds the Pods; only **ready** Pods (readiness probe, lesson 14) receive traffic.
- The list of current Pod addresses is stored in **EndpointSlices**, updated continuously.
- **Types:**

Type	Reachable from	Use
ClusterIP (default)	inside the cluster only	service-to-service traffic (the backend, the database)
NodePort	every node's IP, on a port 30000–32767	simple external access, tests, labs
LoadBalancer	an external load balancer the cloud creates	public services in the cloud (Minikube simulates it with <code>minikube tunnel</code>)

- **DNS names:** `backend-service` (same namespace), `backend-service.services-lab` (from another namespace), `backend-service.services-lab.svc.cluster.local` (full name).

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Selector does not match the Pod labels	no endpoints, timeouts	compare <code>spec.selector</code> with <code>--show-labels</code>

Mistake	Symptom	Fix
<code>targetPort</code> $\neq$ the port the app listens on	endpoints exist, <code>connection refused</code>	<code>targetPort = containerPort</code>
Calling a Pod IP	breaks at the next restart	always call the Service name
Short name from another namespace	<code>bad address</code>	<code>name.namespace</code> or the full name
Pods not ready	no endpoints although labels match	fix the readiness probe (lesson 14)
Expecting a NodePort on <code>localhost</code> with the Docker driver	connection refused	<code>minikube service NAME --url</code>

KEY TAKEAWAYS

- A Service gives Pods a stable IP and DNS name and spreads traffic over the ready Pods it selects.
- `port` is what clients call, `targetPort` is where the Pods listen; EndpointSlices show where traffic goes.
- ClusterIP for inside, NodePort for simple external access, LoadBalancer in the cloud.
- DNS: `name` in the same namespace, `name.namespace` across namespaces.
- Real-world use: every microservice calls the others through Services; most "it cannot connect" problems are a selector, a port or a readiness problem.

Full lesson with every command and its real output: [10-services](#)

Level 6 · Configuration

11 · ConfigMaps

A **ConfigMap** is a Kubernetes object that holds configuration as key–value pairs: a greeting, a log level, a whole configuration file. Pods read it as **environment variables** or as **files** mounted into the container. The image stays the same; only the ConfigMap changes between environments.

An analogy: the image is a printed book, the ConfigMap is the sticky note you put inside the cover ("today's special: soup"). Changing the note does not mean printing a new book.

WHY

If configuration lives inside the image, every change (a URL, a feature switch, a log level) needs a new build, and the same image cannot run in `dev` and `prod` with different settings. The twelve-factor rule "store config in the environment" is what ConfigMaps implement in Kubernetes: build once, configure per environment.

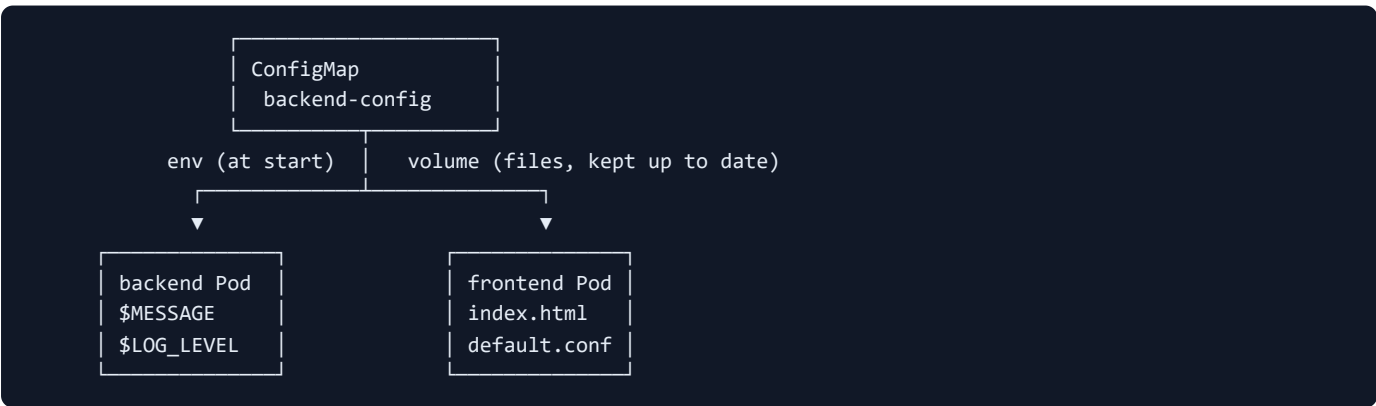
HOW IT WORKS

```
ConfigMap backend-config      Pod
MESSAGE:  "Hello from..."  →env→  $MESSAGE, $LOG_LEVEL      (read once, when the container starts)
LOG_LEVEL: "debug"

ConfigMap frontend-files      Pod
index.html: "<!doctype..." →volume→ /usr/share/nginx/html/index.html  (files; updated in place after a
change)
default.conf: "server {..."  →volume→ /etc/nginx/conf.d/default.conf
```

- **As environment variables** (`env.valueFrom.configMapKeyRef` for one key, `envFrom.configMapRef` for all keys): the values are copied when the container starts. A later change to the ConfigMap is **not** seen until the Pod restarts.
- **As files** (a `configMap` volume): every key becomes a file. The kubelet updates the files when the ConfigMap changes (within about a minute), but the application must reread them.
- A ConfigMap lives in a namespace, and the Pod must be in the same namespace. Size limit: 1 MiB. Not for secrets: that is lesson 12.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Key name differs between Pod and ConfigMap	CreateContainerConfigError, couldn't find key	kubect1 get cm NAME -o jsonpath='{.data}' and compare
ConfigMap in another namespace	configmap "x" not found	create it in the Pod's namespace

Mistake	Symptom	Fix
Expecting env vars to update	the old value stays	<code>kubectl rollout restart deployment/NAME</code>
Mounting a volume over a folder the image needs	the image's other files in that folder disappear	mount single files with <code>items</code> into an empty folder, or use <code>subPath</code>
Putting passwords in a ConfigMap	anyone who can read ConfigMaps sees them	Secrets (lesson 12)

KEY TAKEAWAYS

- ConfigMaps keep configuration out of images: the same image runs everywhere with different settings.
- As variables: copied at container start (restart to pick up changes). As files: kept up to date by the kubelet.
- A wrong key or name gives `CreateContainerConfigError`; `kubectl describe pod` names the missing key.
- Real-world use: URLs of other services, feature switches, log levels and whole configuration files (Nginx, application YAML) for every service of a company, per environment.

Full lesson with every command and its real output: [11-configmaps](#)

12 · Secrets

A **Secret** is like a ConfigMap for sensitive values: passwords, API tokens, keys, certificates. Pods read it the same two ways, as **environment variables** or as **files**. The difference is in how Kubernetes treats it: Secrets are kept apart from ordinary configuration, can be protected separately with RBAC (lesson 24), are not printed by `kubectl describe`, and are only sent to nodes that run a Pod that needs them.

WHY

A password written in an image, a Git repository or a ConfigMap is readable by everyone who can read those. Secrets give credentials their own object, so that access to them can be limited and audited, and so that the same image can use different credentials per environment.

 **A Secret is encoded, not encrypted.** Its values are stored **base64-encoded**, which anyone can decode with one command. Out of the box, Kubernetes also stores Secrets **unencrypted in etcd**, its database; encryption at rest has to be switched on by the cluster administrator. Treat read access to Secrets like access to the passwords themselves, and never commit Secret files with real values to Git.

HOW IT WORKS

Secret db-credentials		Pod
username: YXBw	(base64 of "app")	— env → DB_USER=app, DB_PASSWORD=example-password-change-me
password: ZXhhbXBs...	(base64)	— volume → /etc/db/password (a file, kept in memory on the node)

- `stringData` lets you write plain text in the YAML; Kubernetes encodes it into `data` when it stores the Secret.
- **As a file** (a `secret` volume) is safer than **as a variable**: variables are inherited by child processes and often end up in crash reports and logs. Mounted Secrets live in a memory file system (tmpfs) on the node and are updated when the Secret changes.
- `kubectl describe secret` shows only the sizes of the values; `kubectl get secret -o yaml` shows them encoded.

ARCHITECTURE

```
kubectl apply → API server → etcd (base64; plain on disk unless encryption at rest is enabled)
                        |
                        v
                    only to the node that runs a Pod using it
                        v
                kubelet → tmpfs /etc/db/password + env DB_PASSWORD → container
```

COMMON MISTAKES

Mistake	Symptom	Fix
Wrong Secret name or key	CreateContainerConfigError, secret "x" not found / couldn't find key	compare with <code>kubectl get secret -n NS</code>
Base64-encoding a value twice in <code>data</code>	the application gets gibberish	use <code>stringData</code> , or <code>echo -n value \ base64</code> (no newline)
Committing Secret YAML with real values	the password is in Git history forever	create Secrets from a vault or CI; commit examples only
Thinking base64 is encryption	false sense of security	restrict access (RBAC), enable encryption at rest

Mistake	Symptom	Fix
Expecting env vars to update after a change	the old password stays	mount as a file, or restart the Pods

KEY TAKEAWAYS

- Secrets hold sensitive configuration; Pods read them as variables or (better) as files.
- base64 is an encoding, not encryption; Secrets are unencrypted in etcd unless encryption at rest is enabled.
- Wrong name or key → `CreateContainerConfigError` ; `kubectl describe pod` names what is missing.
- Real-world use: database passwords, API tokens, TLS certificates and registry credentials of every service, often synced from a vault and protected by RBAC.

Full lesson with every command and its real output: [12-secrets](#)

Level 7 · Storage

13 · Storage: volumes, PersistentVolumes, PersistentVolumeClaims and StorageClasses

A container's file system is temporary: when the container is replaced, everything it wrote is gone. A **volume** is a folder that is mounted into the container and lives longer than it.

- **emptyDir**: a scratch folder that lives as long as the **Pod**; containers of the Pod share it.
- **PersistentVolume (PV)**: a piece of real storage in the cluster (a disk, a network share, a folder on a node), which lives independently of any Pod.
- **PersistentVolumeClaim (PVC)**: a request for storage by an application: "I need 1 GiB, read-write". Kubernetes binds the claim to a matching PV.
- **StorageClass**: a recipe for creating PVs automatically when a claim asks for one (**dynamic provisioning**).

An analogy: a PV is a storage unit in a warehouse, the PVC is your rental agreement ("one unit, 1 GiB"), and the StorageClass is the warehouse's catalogue that builds a new unit when nobody has a free one.

WHY

Databases, uploaded files and queues must survive restarts, updates and node failures. Pods are disposable (lessons 08–09 replace them freely); their data must not be. PV and PVC separate the two concerns: developers ask for storage with a claim, without knowing whether it ends up on a cloud disk or a local folder; the cluster provides it.

HOW IT WORKS

```
Pod — uses —> PVC "db-data" (1Gi, RWO) — bound to —> PV (1Gi) — backed by —> real storage
      |                                     ^
      | no PV available?                   | creates
      |                                     |
      |——> StorageClass "standard" —> provisioner (Minikube: a folder on the node)
```

- **Static provisioning**: an administrator creates PVs by hand; claims bind to a matching one.
- **Dynamic provisioning**: the claim names a StorageClass (or uses the default one), and its provisioner creates a PV for it. This is the normal case in clouds (EBS, Azure Disk, Persistent Disk) and in Minikube (`standard`).
- **Access modes**: `ReadWriteOnce` (RWO, one node at a time, the usual for databases), `ReadOnlyMany`, `ReadWriteMany` (many nodes, needs shared storage such as NFS).
- **Reclaim policy**: what happens to the PV when its claim is deleted: `Delete` (the data goes too) or `Retain` (kept for an administrator).

ARCHITECTURE

```
Container
  | mountPath /var/lib/postgresql
  v
Pod — volumes: persistentVolumeClaim db-data
  |
  v
PVC db-data —bound—> PV pvc-... <— created by — StorageClass standard — provisioner k8s.io/minikube-
hostpath
                        |
                        v
                    /tmp/hostpath-provisioner/... on the Minikube node
```

COMMON MISTAKES

Mistake	Symptom	Fix
Unknown <code>storageClassName</code>	PVC <code>Pending</code> , <code>storageclass ... not found</code>	<code>kubect1 get storageclass</code> ; use an existing class
No default StorageClass and no class given	PVC <code>Pending</code> forever, no events	name a class, or mark one default
<code>ReadWriteMany</code> on storage that only supports RWO	<code>Pending</code> or mount errors	RWO for databases; shared file storage for RWX
Data in the container file system or <code>emptyDir</code>	data lost on restart	a PVC
Deleting a PVC still used by a Pod	the PVC stays <code>Terminating</code> (pvc-protection)	delete the Pod (or Deployment) first
<code>RollingUpdate</code> for a database on RWO	the new Pod cannot mount the volume	<code>strategy: Recreate</code> , or a <code>StatefulSet</code> (lesson 19)

KEY TAKEAWAYS

- `emptyDir`: per Pod, temporary, shared between its containers. PVC: storage that outlives Pods.
- Applications request storage with a PVC; a PV satisfies it, made by hand or by a StorageClass's provisioner.
- `Pending` PVC: read its events (wrong class, no default class, nothing that matches).
- Real-world use: every database, message queue and file store on Kubernetes keeps its data on PVCs backed by cloud disks, with StorageClasses chosen for speed and reclaim policy.

Full lesson with every command and its real output: [13-storage](#)

Level 8 · Health checks

14 · Health checks: liveness, readiness and startup probes

A **probe** is a check the kubelet runs against a container, again and again: an HTTP request, a TCP connection or a command. Kubernetes has three, each answering a different question:

startupProbe	"Has the application finished starting?"	until it succeeds, the other two are not run
readinessProbe	"Can the application receive traffic?"	failing → removed from the Service, not restarted
livenessProbe	"Is the application alive?"	failing → the container is killed and restarted

WHY

"The process runs" is not "the application works". A web server can be running while it is still loading, while its database is unreachable, or while it is stuck in a deadlock. Without probes Kubernetes sends traffic to all of these and never restarts the stuck one. With probes, users only reach Pods that can answer, broken Pods heal themselves, and a rolling update stops before a broken version takes over.

HOW IT WORKS

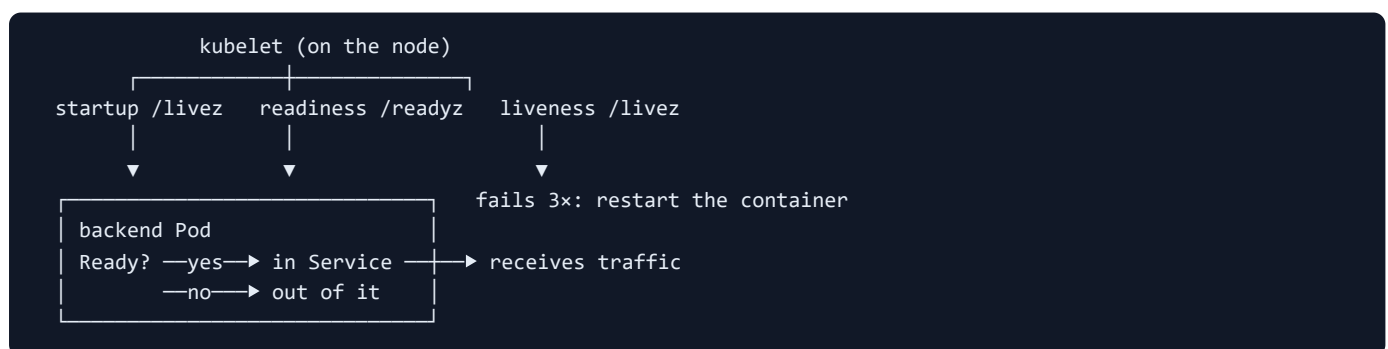
The kubelet on the Pod's node runs each probe every `periodSeconds`. After `failureThreshold` failures in a row:

Probe fails	What Kubernetes does	Typical endpoint
startup	keeps waiting; after <code>failureThreshold × periodSeconds</code> it restarts the container	the same as liveness
readiness	marks the Pod not Ready : the Service stops sending it traffic; the Pod keeps running	checks dependencies (database)
liveness	restarts the container (the restart counter goes up)	checks only the process itself

An HTTP probe succeeds on any status from 200 to 399. Our backend has two endpoints made for this: `/livez` (always 200 while the process runs) and `/readyz` (200 only when its database answers, or when it has none).

Rule of thumb: **liveness must never depend on other services**. If the database is down, restarting every backend does not help; marking them not-ready does.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Liveness checks the database	the database blips, every Pod restarts	liveness = the process only; dependencies in readiness
Liveness without a startup probe on a slow application	restarted before it ever started: <code>CrashLoopBackOff</code>	a startup probe (lab 08)
Probe on the wrong port or path	<code>connection refused</code> / <code>404</code> in events, restarts or never Ready	match the container's real port and path

Mistake	Symptom	Fix
Timeouts too short	flapping between Ready and not Ready under load	<code>timeoutSeconds</code> , <code>failureThreshold</code>
No readiness probe	traffic reaches Pods that are still starting	add one to every Service-backed Pod

KEY TAKEAWAYS

- Startup: finished starting? Readiness: may it get traffic? Liveness: is it alive (else restart)?
- A failing readiness probe removes the Pod from the Service and stalls a rolling update, protecting users.
- Liveness checks only the process; a liveness probe that depends on other services causes restart storms.
- Real-world use: every production Deployment has probes; they make rolling updates safe and recover hung processes without anyone being paged.

Full lesson with every command and its real output: [14-health-checks](#)

Level 9 · Resources and scheduling

15 · Resource requests and limits

Every container can declare how much **CPU** and **memory** it needs:

- **requests**: what the container is guaranteed. The scheduler only places a Pod on a node that still has its requests free.
- **limits**: the most it may use. Above the CPU limit it is slowed down (throttled); above the memory limit the kernel **kills** it (`OOMKilled` , out of memory).

Units: CPU in cores or millicores (`1` = one core, `100m` = 0.1 core); memory in bytes with binary suffixes (`128Mi` = 128 mebibytes, `1Gi`).

WHY

Many Pods share a node. Without requests, the scheduler cannot know whether a node is full and packs too much onto it; without limits, one leaking application can take all the memory and take its neighbours down with it. Requests and limits make a cluster predictable: each application gets what it asked for, and no more than it is allowed.

HOW IT WORKS

```
node: 2 CPU allocatable
├ Pod A requests 1.5 CPU    ✓ scheduled (1.5 of 2 reserved)
├ Pod B requests 0.4 CPU    ✓ scheduled (1.9 of 2 reserved)
└ Pod C requests 0.5 CPU    ✗ Pending: "Insufficient cpu"      (requests decide scheduling)

container with memory limit 128Mi uses 130Mi → kernel kills it → reason OOMKilled, exit code 137
container with cpu limit 250m wants 1 CPU    → slowed down to 0.25 CPU, never killed
```

- The scheduler adds up **requests**, not real usage: a node can be "full" while mostly idle.
- Memory cannot be taken back from a process, so the only answer to too much memory is a kill. CPU can be shared, so too much CPU only means waiting.
- **QoS class**, set from requests and limits: `Guaranteed` (requests = limits for CPU and memory), `Burstable` (some requests), `BestEffort` (none, killed first when a node runs out of memory).

ARCHITECTURE

```
Pod spec: requests / limits
      |           |
      v           v
Scheduler: "does a node still have the requests free?"
yes → bind to the node
no  → Pending (Insufficient ...)

kubelet + kernel (cgroups) on the node:
CPU above the limit → throttle
memory above the limit → OOM kill (exit 137)
```

COMMON MISTAKES

Mistake	Symptom	Fix
Memory limit below the application's real need	<code>OOMKilled</code> , exit 137, <code>CrashLoopBackOff</code>	measure usage, set the limit above the peak
Requests far above real use	Pods <code>Pending</code> with <code>Insufficient cpu/memory</code> while nodes are idle	requests ≈ normal usage
No requests at all	the scheduler overloads nodes; Pods are <code>BestEffort</code> , evicted first	always set requests

Mistake	Symptom	Fix
CPU limit too low	slow responses, probe timeouts, but no errors	raise the CPU limit or remove it (requests only)
Writing 128M instead of 128Mi	slightly different sizes (10^6 vs 2^{20})	use Mi / Gi

KEY TAKEAWAYS

- Requests decide **where** a Pod can run (scheduling); limits decide **how much** it may use.
- Over the memory limit → `OOMKilled`, exit code 137; over the CPU limit → throttled.
- `Pending` with `Insufficient cpu/memory` means requests do not fit on any node.
- Real-world use: sizing requests and limits is daily platform work: it decides cluster cost, stability and how many Pods fit on each node.

Full lesson with every command and its real output: [15-resources](#)

16 · Scheduling basics

Scheduling is choosing a node for a new Pod. The scheduler (lesson 02) filters out the nodes that cannot run it, scores the rest and writes the winner into the Pod. You influence its choice with:

- **node labels** and **nodeSelector**: "only on nodes labelled `disktype=ssd`" (the Pod chooses the node);
- **taints** and **tolerations**: a node says "keep away unless you tolerate `lab=demo`" (the node rejects Pods).

WHY

Nodes are not all equal: some have GPUs or fast disks, some are reserved for one team or for system components, some are being drained for maintenance. Labels and selectors send Pods where they belong; taints keep the wrong Pods away from special nodes.

HOW IT WORKS

```
new Pod (no node) → scheduler
  filter:  enough free requests? (lesson 15)
          nodeSelector labels match?
          taints tolerated?
  score:   spread, least loaded, ...
  bind:    pod.spec.nodeName = winner          nothing fits → Pending + FailedScheduling event
```

- **nodeSelector** is a hard rule: no matching node, no scheduling. (**nodeAffinity** is the richer version: preferences, **In / NotIn** expressions.)
- A taint has a key, a value and an **effect**: **NoSchedule** (new Pods without a toleration are not placed there), **PreferNoSchedule** (avoid if possible), **NoExecute** (also evict running Pods). A toleration only *allows* a Pod onto a tainted node; it does not attract it there.
- Control-plane nodes in real clusters carry the taint `node-role.kubernetes.io/control-plane:NoSchedule`; Minikube's single node does not, so it can run your Pods.

ARCHITECTURE

```
node minikube
labels:  kubernetes.io/hostname=minikube, disktype=ssd (added in the lab)
taints:  lab=demo:NoSchedule (added in Break It)
  ▲      ▲
  |      |
  | nodeSelector: disktype=ssd   | tolerations: lab=demo:NoSchedule
  | Pod fast-disk                | Pod tolerant
```

COMMON MISTAKES

Mistake	Symptom	Fix
nodeSelector label nobody has (or a typo)	Pending, "didn't match Pod's node affinity/selector"	<code>kubectl get nodes --show-labels</code> ; label the node or fix the selector
Expecting a toleration to attract a Pod	the Pod lands on other nodes	combine a toleration with a nodeSelector
Forgetting the effect in <code>kubectl taint</code>	error: unknown taint spec	<code>key=value:NoSchedule</code>
Removing a label and expecting Pods to move	nothing happens	delete the Pods; the Deployment recreates them where they fit
Tainting the only node	every new Pod Pending	remove the taint: <code>kubectl taint node NAME key=value:EFFECT-</code>

KEY TAKEAWAYS

- The scheduler filters nodes by free requests, `nodeSelector` labels and taints, then picks one.
- `nodeSelector` pulls a Pod to labelled nodes; a taint pushes Pods away unless they tolerate it.
- `Pending` + `FailedScheduling` event = read the message: it names the rule no node satisfied.
- Real-world use: GPU and high-memory node pools, nodes reserved for a team or for system agents, and draining nodes for maintenance (`kubect1 drain` adds a `NoSchedule` taint).

Full lesson with every command and its real output: [16-scheduling](#)

Level 10 · Jobs, CronJobs and DaemonSets

17 · Jobs and CronJobs

A **Job** runs a task **until it succeeds, then stops**: a database migration, a report, an import. A **CronJob** creates a Job **on a schedule**: every night at 2:00, every minute, every Monday.

The analogy: a Deployment is a shop that must stay open all day; a Job is a delivery that is done once the parcel arrives; a CronJob is the delivery round that happens every morning.

WHY

A Deployment restarts its Pods forever: if your backup script finishes and exits, the Deployment would start it again and again. Some work has an end. A Job knows the difference between "finished successfully" and "crashed", retries failures a limited number of times, and keeps the result (the Pod and its logs) for you to read.

```
Deployment → long-running application, always N copies running
Job        → one-time task: run until it succeeds, then stop
CronJob    → a Job on a schedule
```

HOW IT WORKS

The Job controller creates Pods from the Job's template and watches how they end:

- exit code 0 → one **completion**; when `completions` are reached, the Job is `Complete`;
- non-zero exit → a failure; a new Pod is created, up to `backoffLimit` retries, with a growing delay (10 s, 20 s, 40 s...); after that the Job is `Failed` with the reason `BackoffLimitExceeded`.

`restartPolicy` must be `Never` (a failed Pod is replaced by a new one) or `OnFailure` (the container is restarted inside the same Pod); never `Always`, which is for long-running Pods.

The CronJob controller checks the `schedule` every few seconds and creates a Job (named `CRONJOB-<number>`) when it is due. Old Jobs are deleted according to the history limits.

ARCHITECTURE

```
CronJob "report" — every minute → Job report-29365432 → Pod report-29365432-x7k2p → exit 0 → Complete
                  — every minute → Job report-29365433 → Pod ... → exit 0 → Complete

Job "import" → Pod #1 exit 1 → Pod #2 exit 1 → Pod #3 exit 1 → backoffLimit 2 reached → Failed
```

COMMON MISTAKES

Mistake	Symptom	Fix
<code>restartPolicy:</code> <code>Always</code> in a Job	Unsupported value: "Always" when applying	<code>Never</code> or <code>OnFailure</code>
A Job's command that never exits	the Job runs forever, <code>COMPLETIONS 0/1</code>	the task must end; add <code>activeDeadlineSeconds</code> as a safety limit
Re-applying a changed Job	<code>field is immutable</code>	delete the Job and create it again (Jobs are run-once)
Wrong cron syntax or time zone	runs at the wrong time, or never	check with <code>kubectl get cronjob</code> (LAST SCHEDULE); set <code>timeZone: "Europe/Berlin"</code> if needed
No history limits / TTL	thousands of finished Jobs and Pods pile up	<code>successfulJobsHistoryLimit</code> , <code>failedJobsHistoryLimit</code> , <code>ttlSecondsAfterFinished</code>

KEY TAKEAWAYS

- Job = run to completion, retry failures up to `backoffLimit`; CronJob = a Job on a cron schedule.

- `completions` and `parallelism` split work across Pods.
- A failed Job's Pods keep their logs: `kubect1 logs -l job-name=NAME` .
- Real-world use: database migrations before a release, nightly backups and reports, cleanup tasks, batch imports.

Full lesson with every command and its real output: [17-jobs-cronjobs](#)

18 · DaemonSets

A **DaemonSet** runs **one Pod on every suitable node** of the cluster. Add a node, and it gets its Pod automatically; remove a node, and its Pod goes with it.

The analogy: a Deployment is "three cashiers, wherever there is room"; a DaemonSet is "one security guard in every building", however many buildings there are.

WHY

Some work belongs to the **node**, not to the application: collecting the node's log files, measuring its CPU and disk, configuring its network. That work must happen exactly once per node. With a Deployment you would have to keep `replicas` equal to the number of nodes and hope the scheduler spreads them evenly; a DaemonSet guarantees it.

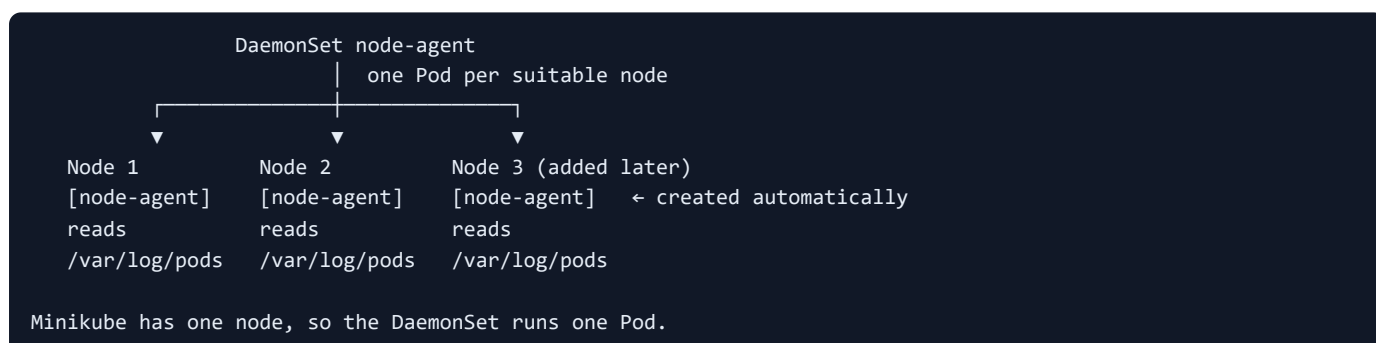
You have already met DaemonSets: `kube-proxy` and `calico-node` (lesson 02) run on every node for exactly this reason.

HOW IT WORKS

The DaemonSet controller looks at every node, checks whether the node is **suitable** (it matches the `nodeSelector`, its taints are tolerated: lesson 16), and makes sure exactly one Pod of the DaemonSet runs there. There is no `replicas` field: the number of Pods is the number of suitable nodes.

A node agent often needs to see the node itself, so it mounts a folder of the node with a **hostPath** volume (lesson 13 covers volumes): here, read-only, the folder where the node keeps the logs of every Pod.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
A <code>nodeSelector</code> no node matches	<code>DESIRED 0</code>	label the nodes (<code>kubectl get nodes -L KEY</code> shows them) or fix the selector
Nodes with taints (control plane, GPU nodes)	no Pod on those nodes	add <code>tolerations</code> to the template (lesson 16)
Heavy agents with no limits	every node loses CPU/memory to the agent	small <code>requests</code> and <code>limits</code>
Writable <code>hostPath</code> mounts	an agent can damage the node	<code>readOnly: true</code> ; mount only what is needed

KEY TAKEAWAYS

- DaemonSet = exactly one Pod on every suitable node; no `replicas`.
- "Suitable" = matches the `nodeSelector` and tolerates the node's taints.
- `DESIRED 0` is not an error: no node matches.
- Real-world use: log collectors (Fluent Bit), monitoring agents (node-exporter), network plugins (Calico, Cilium), storage drivers, security agents.

Full lesson with every command and its real output: [18-daemonsets](#)

Level 11 · Stateful applications

19 · StatefulSets

A **StatefulSet** runs Pods that each have a **stable identity**: a fixed name (`notes-0` , `notes-1` , `notes-2`), a fixed DNS name, and **their own disk** that follows them when they are replaced. Pods are created in order and removed in reverse order.

The analogy: Deployment Pods are like hotel guests: any room will do, and nobody minds who is in which room. StatefulSet Pods are like employees with a desk and a locker: `notes-1` always comes back to locker `data-notes-1`.

WHY

Deployments treat Pods as **interchangeable**: random names, shared or no storage, any order. That is perfect for a stateless web server. It breaks for databases and clustered systems (PostgreSQL replicas, Kafka, Elasticsearch, etcd): each member keeps its own data, the others must find it by a stable name, and the first member often has to start before the others join.

```
Deployment    → interchangeable Pods: web-7f9c-abcde, web-7f9c-xyz12, same template, no own disk
StatefulSet   → stable identity and storage: notes-0, notes-1, notes-2, each with data-notes-N
```

HOW IT WORKS

- **Names:** `<statefulset>-<ordinal>` , from 0 up. A replaced Pod gets the **same** name.
- **Order:** with the default `podManagementPolicy: OrderedReady` , `notes-1` is created only when `notes-0` is ready; scaling down removes the highest number first.
- **Storage:** `volumeClaimTemplates` creates one PersistentVolumeClaim per Pod (`data-notes-0` , ...), and the Pod with that number always mounts it. The PVCs are **not** deleted with the Pods or the StatefulSet: the data survives.
- **Network identity:** a **headless Service** (`clusterIP: None`) gives each Pod a DNS name, `notes-0.notes.<namespace>.svc.cluster.local` , so the members can address each other directly.

ARCHITECTURE

```
headless Service "notes" (clusterIP: None)
notes-0.notes  notes-1.notes  notes-2.notes  ← one DNS name per Pod
|              |              |
StatefulSet notes → notes-0 → notes-1 → notes-2      created in this order
|              |              |
PVC data-notes-0 data-notes-1 data-notes-2      one disk each, kept forever
|              |              |
PV (standard StorageClass, Minikube's hostpath provisioner)
```

COMMON MISTAKES

Mistake	Symptom	Fix
No headless Service, or a different <code>serviceName</code>	the Pods run but <code>notes-0.notes</code> does not resolve	create the headless Service named in <code>serviceName</code>
A StorageClass that does not exist	PVC and first Pod <code>Pending</code>	<code>kubectl get storageclass</code> ; fix the template, delete the StatefulSet and its PVCs
Expecting PVCs to go away with the StatefulSet	disks pile up; old data reappears on re-create	delete PVCs explicitly when the data is really no longer needed
Changing <code>volumeClaimTemplates</code> with <code>apply</code>	Forbidden: updates to <code>statefulset spec ...</code> are forbidden	delete and re-create (with <code>--cascade=orphan</code> to keep the Pods running)

Mistake	Symptom	Fix
Using a StatefulSet for a stateless app	slower rollouts, ordering for nothing	use a Deployment

KEY TAKEAWAYS

- StatefulSet = stable names (`name-0...`), ordered start/stop, one PVC per Pod that survives the Pod.
- A headless Service (`clusterIP: None`) gives each Pod a DNS name:
`pod.service.namespace.svc.cluster.local` .
- PVCs outlive Pods and the StatefulSet: deleting data is always an explicit step.
- Real-world use: databases and clustered systems (PostgreSQL, MySQL, Kafka, Redis, Elasticsearch, etcd), usually installed through an operator or a Helm chart that uses StatefulSets underneath.

Full lesson with every command and its real output: [19-statefulsets](#)

Level 12 · Ingress and networking

20 · Ingress

An **Ingress** is a set of HTTP routing rules: "requests for `app.local` go to the frontend Service, requests for `shop.local/api` go to the backend Service". An **ingress controller** is the program (a reverse proxy running in the cluster) that reads those rules and actually routes the traffic.

The analogy: Services are the departments of a company; the Ingress is the reception desk with one public entrance, reading the visitor's request ("I am here for accounting") and sending them to the right floor.

WHY

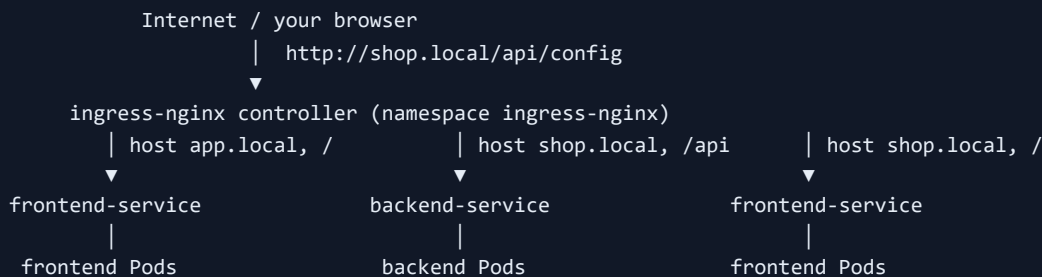
Exposing every application with its own `NodePort` or `LoadBalancer` Service (lesson 10) does not scale: one random high port or one paid cloud load balancer **per application**, no host names, no shared TLS certificate, no paths. Web traffic wants one entrance (ports 80/443) and rules by **host name** and **path**. That is the Ingress.

HOW IT WORKS

```
Ingress object (rules) —read by—> ingress controller (a Deployment of reverse-proxy Pods, e.g. ingress-nginx)
                                     | turns the rules into its proxy configuration
client —HTTP Host: shop.local /api/x —> controller —> backend-service —> backend Pods
```

- The Ingress object alone does nothing: without a controller, nobody reads it.
- A cluster can run several controllers; `ingressClassName` says which one handles an Ingress. Minikube's addon installs **ingress-nginx** with the class `nginx`. (The ingress-nginx project was retired upstream in 2026; other controllers such as Traefik, and the newer **Gateway API**, use the same ideas: host and path rules to Services.)
- The controller itself is reached from outside through a Service: on Minikube a `NodePort` on the node.
- Rules match the `Host` header first, then the **longest** matching path.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
No ingress controller installed	Ingress objects accepted, nothing answers	install one (<code>minikube addons enable ingress</code>)
Missing or wrong <code>ingressClassName</code>	the Ingress gets no address, requests 404	<code>kubectl get ingressclass</code> ; set the class
Wrong Service name or port	<code>503</code> from the controller; <code><error: services ... not found></code> in describe	match <code>kubectl get svc</code>
Service in another namespace	<code>not found</code>	an Ingress routes only to Services in its own namespace
Testing without the right <code>Host</code> header	<code>404</code> from the default backend	send the host name (<code>curl -H 'Host: ...'</code> , hosts file, DNS)

Mistake	Symptom	Fix
Expecting a Pod-level fix to show at once	old errors for a few seconds	the controller reloads its config; retry

KEY TAKEAWAYS

- Ingress = HTTP routing rules (host, path → Service); the ingress controller does the routing.
- One entrance for many applications: no NodePort per app.
- 503 from the controller → a backend Service is missing or has no ready Pods; 404 → no rule matched.
- Real-world use: every web application on Kubernetes is published through an Ingress or Gateway, usually with TLS, behind one cloud load balancer.

Full lesson with every command and its real output: [20-ingress](#)

21 · Kubernetes networking

The Kubernetes **networking model** is a short list of promises every cluster keeps, whatever network plugin it uses:

```
Pod → Pod      every Pod has its own IP; every Pod can reach every other Pod directly, without NAT
Pod → Service   a Service has a stable virtual IP and a DNS name; traffic is spread over its ready Pods
Pod → Internet  Pods can reach the outside world (through the node)
External → Service a NodePort, LoadBalancer or Ingress lets outside traffic in
```

WHY

Every "it cannot connect" problem is one of these four paths. Knowing which path a request takes, and which part of the cluster is responsible for it, turns guessing into a short investigation: is it the Pod, the Service, DNS, a NetworkPolicy (lesson 22), or the way in from outside?

HOW IT WORKS

Piece	What it does	Where
CNI plugin (Calico here)	gives every Pod an IP from the Pod network (<code>10.244.x.x</code>) and routes between them	a DaemonSet (<code>calico-node</code>)
kube-proxy	turns every Service IP into rules that forward to the Pods behind it	a DaemonSet on every node
CoreDNS	answers names: <code>web</code> → the Service IP; <code>web.net-lab.svc.cluster.local</code>	Pods in <code>kube-system</code>
NodePort / LoadBalancer / Ingress	the doors from outside	Services, ingress controller (lesson 20)

Each Pod's `/etc/resolv.conf` points at CoreDNS and lists **search domains** (`<namespace>.svc.cluster.local` , ...), so a short name like `web` is completed with the Pod's **own** namespace.

ARCHITECTURE

```
outside → node:30080 (NodePort) —┐
                                ▼
client Pod 10.244.0.20 — http://web → CoreDNS: web = 10.103.0.10 (Service IP)
                                | kube-proxy rules
                                ▼
┌ Pod → Pod (direct, no NAT)
├ web Pod 10.244.0.11      web Pod 10.244.0.12
└ Internet (through the node's address)
```

COMMON MISTAKES

Mistake	Symptom	Fix
Hard-coding a Pod IP	works until the Pod is replaced	always use a Service name
Short name across namespaces	<code>bad address / Name or service not known</code>	<code>service.namespace</code>
Expecting to <code>ping</code> a Service IP	no reply	Service IPs are virtual: they forward TCP/UDP ports, not ICMP; test with the port
Using <code>targetPort</code> from outside	connection refused	from outside use the <code>nodePort</code> (or Ingress), inside the Service <code>port</code>
Using <code>localhost</code> to reach another Pod	connection refused	<code>localhost</code> is the Pod itself (shared only between containers of one Pod, lesson 06)

KEY TAKEAWAYS

- Every Pod has an IP and reaches every other Pod directly; Pod IPs change.
- Services give a stable IP and DNS name; kube-proxy forwards to the ready Pods.
- Short names resolve in the Pod's own namespace; use `name.namespace` across namespaces.
- NodePort / LoadBalancer / Ingress are the ways in from outside.
- Real-world use: "service A cannot reach service B" is among the most common incidents; walking the four paths finds the broken step quickly.

Full lesson with every command and its real output: [21-networking](#)

22 · NetworkPolicies

A **NetworkPolicy** is a firewall rule for Pods: "the database accepts connections **only** from the backend, on port 5432". It selects Pods by label and lists who may talk to them (ingress) or whom they may talk to (egress).

The analogy: by default every office in the building has its door open. NetworkPolicies are badge readers: once a door has one, only the badges listed may enter.

WHY

Lesson 21's model says every Pod can reach every Pod. That is convenient and dangerous: if an attacker takes over the frontend (the part exposed to the Internet), they can connect straight to the database. NetworkPolicies enforce the architecture you drew: frontend → backend → database, and nothing else.

```
frontend → backend    allowed
backend  → database    allowed
frontend × database    denied
```

HOW IT WORKS

- A Pod is **isolated** for ingress as soon as **any** policy with `policyTypes: [Ingress]` selects it. From then on, only traffic allowed by some policy gets in. Pods selected by no policy stay open.
- Policies only **add** permissions; there is no "deny" rule. The usual pattern: one **default-deny** policy selecting every Pod (`podSelector: {}`), then small **allow** policies.
- `from` can select Pods by label (`podSelector`), namespaces (`namespaceSelector`) or IP ranges (`ipBlock`).
- The **network plugin** enforces them. This course's cluster runs Calico, which does; a plugin without NetworkPolicy support accepts the objects and silently ignores them (that is why lesson 03 starts Minikube with `--cni=calico`).

ARCHITECTURE

```
namespace netpol-lab, policies: default-deny-ingress + two allow rules
```

```
frontend — TCP 8080 ✓ —> backend — TCP 5432 ✓ —> database
|                                     (allow-frontend-to-backend) (allow-backend-to-database)
|_____ TCP 5432 × _____| (no rule allows it)
```

COMMON MISTAKES

Mistake	Symptom	Fix
A network plugin without NetworkPolicy support	policies exist, nothing is blocked	use Calico/Cilium (<code>minikube start --cni=calico</code>)
A label typo in <code>podSelector</code> or <code>from</code>	timeouts, or nothing protected	<code>kubectl get pods --show-labels; describe networkpolicy</code>
Default-deny egress without allowing DNS	every name lookup fails	allow UDP/TCP 53 to <code>kube-system</code> CoreDNS
Forgetting the port	traffic still blocked	list the destination port in <code>ports</code>
Assuming policies are ordered	"deny" rules expected	policies only allow; isolation comes from being selected

KEY TAKEAWAYS

- NetworkPolicies are label-based firewall rules for Pods; selected Pods accept only what some policy allows.
- Default-deny first, then explicit allow rules for the paths of the architecture.
- A blocked connection usually times out; check policies and labels.

- They need a network plugin that enforces them (Calico in this course).
- Real-world use: limiting the damage of a compromised Pod, separating teams and environments in shared clusters, meeting security requirements ("only the API may reach the database").

Full lesson with every command and its real output: [22-networkpolicies](#)

Level 13 · Security and RBAC

23 · ServiceAccounts

A **ServiceAccount** is an identity for a **workload**: a Pod that talks to the Kubernetes API (to list Pods, read a ConfigMap, create a Job) does it **as** a ServiceAccount. Kubernetes gives the Pod a short-lived token proving that identity.

The analogy: people get a personal badge (a **user** account); the delivery robot in the building gets its own badge too, with its own name, so the doors know it is the robot and not a person.

WHY

Some applications need the Kubernetes API: a CI runner creating Pods, a monitoring agent reading nodes, an operator managing a database, the ingress controller reading Ingresses (lesson 20). Each needs an identity, so that permissions (lesson 24) can be given to **that** application only, and so the audit log shows which application did what.

	User	ServiceAccount
Who	a person (you, a colleague)	a workload (a Pod)
Created by	outside Kubernetes (certificates, an identity provider)	Kubernetes itself: <code>kind: ServiceAccount</code>
Scope	the cluster	one namespace
Name in permissions	<code>developer</code>	<code>system:serviceaccount:<namespace>:<name></code>

HOW IT WORKS

- Every namespace has a ServiceAccount called `default`; a Pod that names none uses it.
- `spec.serviceAccountName` chooses another one. The kubelet mounts a token, the cluster's CA certificate and the namespace into `/var/run/secrets/kubernetes.io/serviceaccount/`; the token is short-lived and renewed automatically.
- The API server is reachable from every Pod at `https://kubernetes.default.svc`.
- A ServiceAccount starts with **no permissions** (beyond discovering the API): identity is not authorization. RoleBindings (lesson 24) grant permissions.
- `automountServiceAccountToken: false` keeps the token out of Pods that never call the API.

ARCHITECTURE

```
Pod "reporter" (serviceAccountName: reporter)
/var/run/secrets/kubernetes.io/serviceaccount/
token ca.crt namespace
|
| Authorization: Bearer <token>
|
▼
API server — authentication: "this is system:serviceaccount:sa-lab:reporter"
            — authorization (RBAC, lesson 24): "may it list Pods?" → no binding → 403 Forbidden
```

COMMON MISTAKES

Mistake	Symptom	Fix
Naming a missing ServiceAccount	Deployment <code>0/1</code> , no Pods, <code>FailedCreate</code> on the ReplicaSet	create it in the same namespace
Expecting a new ServiceAccount to have permissions	<code>403 Forbidden</code>	bind a Role to it (lesson 24)

Mistake	Symptom	Fix
Giving the <code>default</code> ServiceAccount permissions	every Pod in the namespace gets them	one ServiceAccount per application
Mounting tokens everywhere	a stolen Pod = a stolen credential	<code>automountServiceAccountToken:</code> <code>false</code>
Copying a token into a ConfigMap or an image	a long-lived secret leaks	use the mounted, auto-rotated token

KEY TAKEAWAYS

- ServiceAccount = a namespaced identity for workloads; users are for people.
- Pods get a short-lived token at `/var/run/secrets/kubernetes.io/serviceaccount/token` ; the API is at `https://kubernetes.default.svc` .
- Identity is not permission: a new ServiceAccount gets `403` until a RoleBinding allows it (lesson 24).
- Real-world use: CI/CD runners, operators, monitoring agents, ingress controllers and any app calling the API each run with their own ServiceAccount and the smallest permissions.

Full lesson with every command and its real output: [23-serviceaccounts](#)

24 · RBAC

RBAC (role-based access control) decides **who may do what** in a cluster. A **Role** lists permissions ("get, list and watch Pods"); a **RoleBinding** gives a Role to someone (a user, a group, a ServiceAccount). Their cluster-wide versions, **ClusterRole** and **ClusterRoleBinding**, cover objects without a namespace (nodes) or all namespaces at once.

The analogy: a Role is a job description ("may read the patient files"), a RoleBinding is the signed contract that gives that job to a person, in one department.

WHY

Without authorization, anyone who can reach the API can delete production. Teams need rules like: developers may look at Pods and logs in their namespace, but not delete them; the CI system may deploy to `staging` only; a monitoring agent may read nodes everywhere. RBAC expresses exactly that, and `kubectl auth can-i` checks it.

HOW IT WORKS

Every API request is checked in two steps:

```
1. authentication  "who are you?"  certificate, token → user "developer", or system:serviceaccount:ns:name
2. authorization  "may you?"      RBAC: is there a binding that gives this subject a role that allows
                                   VERB on RESOURCE (in NAMESPACE)?  yes → allowed, no → 403 Forbidden
```

- Permissions are **additive** and **deny by default**: no matching rule = forbidden. There are no "deny" rules.
- A rule = `apiGroups` ("" for the core group: Pods, Services...; `apps` for Deployments...) + `resources` + `verbs` (`get` , `list` , `watch` , `create` , `update` , `patch` , `delete`).
- Role + RoleBinding = permissions in **one** namespace. ClusterRole + ClusterRoleBinding = everywhere, and for cluster-scoped objects (nodes, namespaces, PersistentVolumes). A RoleBinding may also point at a ClusterRole, to reuse a common role in one namespace (Kubernetes ships `view` , `edit` , `admin`).
- Kubernetes has no user database: users come from certificates or an identity provider. In this lab you **impersonate** a user with `--as developer` (allowed for the cluster administrator you are on Minikube).

ARCHITECTURE

```
User "developer" └─ subjects ──> RoleBinding developer-views-pods (namespace rbac-lab)
SA "reporter" ───┘               | roleRef
                               ▼
                             Role pod-viewer: pods, pods/log → get, list, watch

User "developer" ─ ClusterRoleBinding ─> ClusterRole course-node-viewer: nodes → get, list

developer: get pods in rbac-lab ✓  delete pods ✗  get nodes ✓  get pods in kube-system ✗
```

COMMON MISTAKES

Mistake	Symptom	Fix
A typo in <code>roleRef</code> or subject name	<code>no</code> / <code>403</code> , no error when applying	<code>kubectl describe rolebinding</code> ; compare with <code>kubectl get roles</code>
Binding in the wrong namespace	allowed in one namespace, forbidden in another	a RoleBinding works only in its own namespace
Wrong <code>apiGroups</code>	<code>403</code> for Deployments	Deployments are in <code>apps</code> , Ingresses in <code>networking.k8s.io</code> (<code>kubectl api-resources</code> shows the group)

Mistake	Symptom	Fix
Using a Role for nodes	cannot list resource "nodes" ... at the cluster scope	ClusterRole + ClusterRoleBinding
Giving cluster-admin "to make it work"	everything works, including deleting everything	grant only the verbs and resources needed
Granting * verbs or * resources	far more access than intended	list them explicitly

KEY TAKEAWAYS

- Authentication says who you are; RBAC authorization says what you may do; the default is deny.
- Role + RoleBinding = one namespace; ClusterRole + ClusterRoleBinding = cluster-wide and cluster-scoped objects.
- `kubectl auth can-i VERB RESOURCE -n NS --as SUBJECT` answers every RBAC question.
- Real-world use: giving teams access to their namespaces, limiting CI/CD to the environments it deploys to, granting agents read-only access, passing security audits.

Full lesson with every command and its real output: [24-rbac](#)

Level 14 · Scaling and autoscaling

25 · Scaling

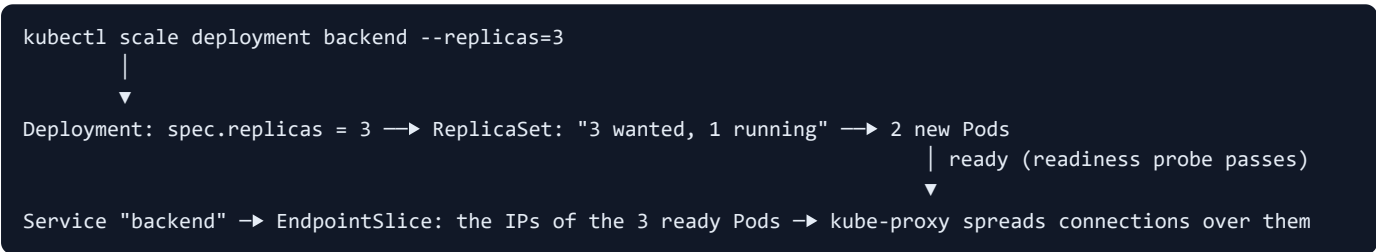
Scaling means changing how many copies (**replicas**) of an application run. With Kubernetes you do not start or stop containers yourself: you change one number on the Deployment, `replicas`, and the Deployment's ReplicaSet creates or deletes Pods until reality matches it. The Service in front of the Pods notices the change on its own and spreads requests over however many Pods are ready.

Think of a supermarket: when the queues get long, the manager opens more tills; the customers do not need to know which till is open, they just join "the checkout".

WHY

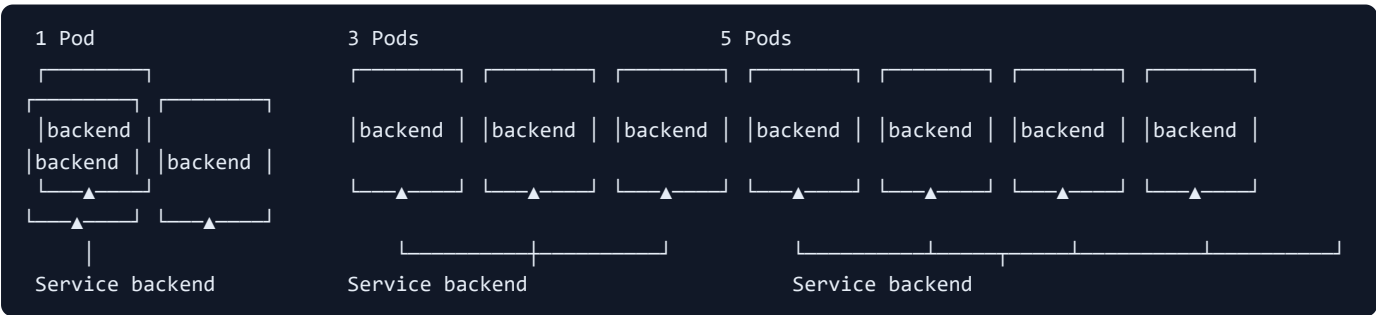
One Pod has limits: one container's CPU and memory, and when it restarts, nobody is served. More replicas mean more capacity (more requests per second) and availability (one Pod can die while the others keep serving). Fewer replicas save resources when traffic is low. Scaling by hand is the first step; lesson 26 lets Kubernetes do it automatically.

HOW IT WORKS



- `kubectl scale` only changes `spec.replicas`; the rest is the reconciliation you saw in lessons 08–09.
- A new Pod receives traffic only once its readiness probe passes (lesson 14): scaling never sends requests to a Pod that is still starting.
- The Service does not count Pods: its **EndpointSlice** lists the addresses of the ready Pods matching its selector, and kube-proxy picks one of them for every new connection.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
<code>kubectl scale</code> , then <code>kubectl apply</code> of the old file	replicas jump back to the file's value	change <code>replicas:</code> in the file (the source of truth)
Scaling a Deployment that an HPA manages	the HPA overrides your number within seconds (lab 13)	change the HPA's <code>minReplicas</code> / <code>maxReplicas</code> instead
Scaling up without resource requests	Pods crowd one node, or the node runs out of memory	set requests (lesson 15) so the scheduler can plan
Scaling to 0 "temporarily"	<code>Connection refused</code> for every client	check <code>kubectl get deploy</code> first when a Service refuses connections

Mistake	Symptom	Fix
Expecting equal traffic per Pod	one Pod gets more	spreading is per connection and random; long-lived connections stick to one Pod

KEY TAKEAWAYS

- Scaling = changing `spec.replicas` ; the ReplicaSet creates or deletes Pods to match.
- The Service follows automatically: its EndpointSlice lists every ready Pod, kube-proxy spreads connections.
- `Connection refused` on a Service usually means no endpoints: 0 replicas, a wrong selector, or no ready Pod.
- Real-world use: scaling up before a known traffic peak, scaling down at night, and running enough replicas to survive the loss of a Pod or a node.

Full lesson with every command and its real output: [25-scaling](#)

26 · Horizontal Pod Autoscaler

A **HorizontalPodAutoscaler** (HPA) changes a Deployment's `replicas` for you, based on a measurement: usually CPU usage. When the Pods work hard it adds Pods; when they are idle it removes them, between a minimum and a maximum you choose. "Horizontal" means more Pods, as opposed to "vertical", bigger Pods.

It is the supermarket manager of lesson 25 with a rule written down: "open a till whenever the queues are longer than 5 people; close one when they are shorter; never fewer than 1 or more than 5 tills".

WHY

Traffic changes during the day: busy at lunch, quiet at night, a spike when a newsletter goes out. Scaling by hand means someone watches graphs and types `kubectl scale`, and is always too late or wastes capacity. The HPA reacts within a minute, around the clock, and only uses the resources the load needs.

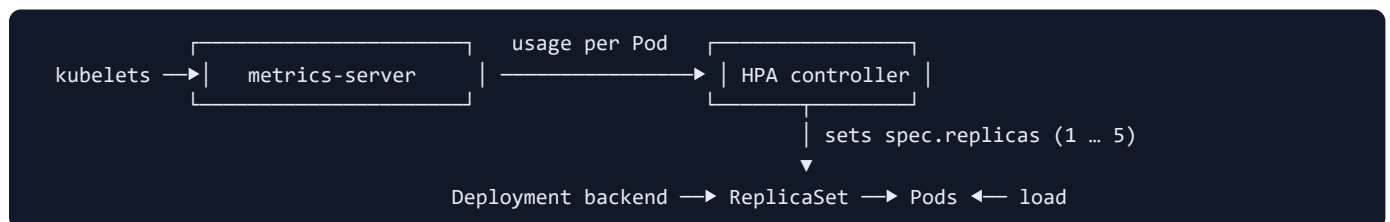
HOW IT WORKS

CPU increases → HPA detects load → more Pods → CPU per Pod decreases → (later) Pods scale down

1. **metrics-server** (a cluster add-on) collects every Pod's CPU and memory usage from the kubelets, every 15 s.
2. The HPA controller (in the controller manager) compares the average usage with the **target**, as a percentage of what each Pod **requests**: request 100m, use 80m → 80 %.
3. It computes `desired = ceil(current replicas × current % / target %)`, within `minReplicas` ... `maxReplicas`, and sets the Deployment's `replicas`.
4. Scaling up happens quickly; scaling down waits for a **stabilization window** (5 minutes by default), so a short pause in traffic does not remove Pods that are needed a moment later.

Because utilization is relative to the **request**, an HPA cannot work for containers without CPU requests: that is this lesson's Break It.

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
No metrics-server	TARGETS <unknown>, failed to get cpu utilization: unable to get metrics	install it (<code>minikube addons enable metrics-server</code>)
No CPU request	<unknown>, missing request for cpu	set <code>resources.requests.cpu</code> on every container
Scaling by hand an HPA-managed Deployment	the number jumps back	change the HPA's min/max
<code>replicas</code> in the Deployment file re-applied	brief jumps to the file's value	remove <code>replicas</code> from the manifest when an HPA manages it
Expecting instant scale-down	Pods stay for minutes after the load	that is the stabilization window (300 s by default), on purpose

KEY TAKEAWAYS

- An HPA sets `replicas` from a measurement, between min and max; metrics-server provides CPU and memory.
- CPU utilization is relative to the **request**: no request, no autoscaling (`<unknown>`).
- Scale-up is fast; scale-down waits for the stabilization window (default 5 minutes).
- Real-world use: web services and APIs whose traffic varies over the day scale with an HPA; it is also the first step towards cost control in the cloud.

Full lesson with every command and its real output: [26-hpa](#)

Level 15 · Troubleshooting and Helm

27 · Troubleshooting Kubernetes

Troubleshooting is finding out **why** the cluster does not do what you asked, with a method instead of guesses. Kubernetes is unusually good at telling you what is wrong: every object has a status, every important step leaves an **event**, every container keeps its **logs**, and you can step **inside** a running Pod. The skill is knowing which of these to read first.

A doctor does not start with surgery: they ask what hurts, take your temperature, look at the test results, and only then decide. Here the symptoms are `kubectl get`, the temperature is `kubectl describe` and the events, the test results are the logs, and the examination is `kubectl exec` and `kubectl debug`.

WHY

Something always breaks: a typo in a label, an image tag that does not exist, a memory limit that is too small, a policy that blocks a port. Copying the error text into a search engine finds other people's problems. Following the same questions every time finds yours, usually in a few minutes. Lab 12 gives you fourteen real problems to practise on; this lesson gives you the method and the tools.

HOW IT WORKS

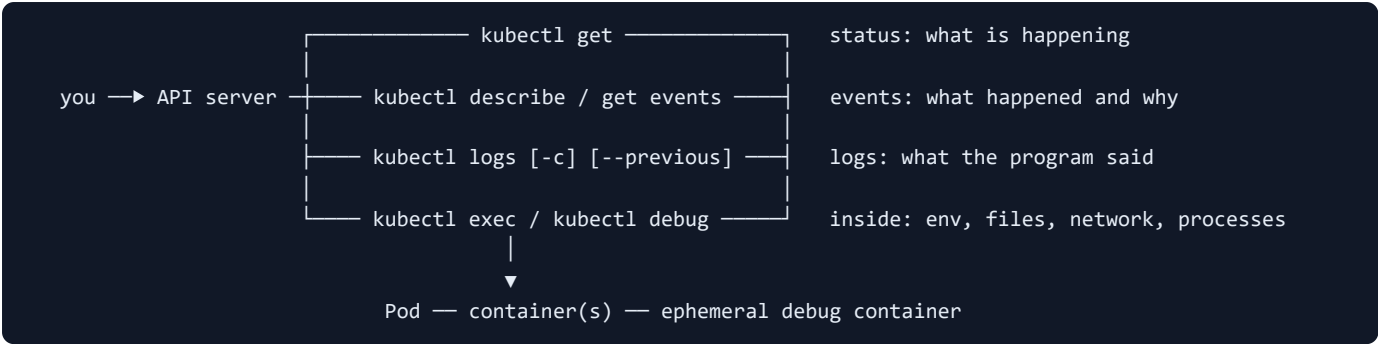
The troubleshooting mindset, in order:

```
What is broken?           "the backend does not answer"
  ↓
What should happen?       "3 Pods ready behind the Service"
  ↓
What actually happened?    kubectl get: 0/1 READY, CrashLoopBackOff, Pending, ...
  ↓
Which Kubernetes object controls it? Pod? Deployment? Service? PVC? Ingress? NetworkPolicy?
  ↓
Inspect the object         kubectl describe / kubectl get -o yaml
  ↓
Check events               kubectl get events, the Events section of describe
  ↓
Check logs                 kubectl logs (--previous, -c)
  ↓
Check configuration        env, ConfigMaps, Secrets, probes, selectors, resources
  ↓
Find the root cause → Fix → Verify    and verify with the same command that showed the problem
```

The status of a Pod already narrows it down:

You see	It means	Look at
Pending	not scheduled, or waiting for a volume	events (FailedScheduling), the PVC
ContainerCreating for long	image pull, volume mount, network setup	events
ErrImagePull / ImagePullBackOff	the image cannot be downloaded	the Failed to pull image event
CreateContainerConfigError	a ConfigMap, Secret or key is missing	the Error: event
CrashLoopBackOff	the container keeps exiting	kubectl logs --previous, exit code
Running, 0/1 READY	the readiness probe fails	the Unhealthy event
OOMKilled, exit 137	over the memory limit	limits, kubectl top
everything Running, still no answer	Service, DNS, Ingress, NetworkPolicy	endpoints, wget from a Pod

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Reading <code>kubectl logs</code> of a restarted container	empty or unrelated logs	<code>--previous</code>
Forgetting <code>-c</code> on a multi-container Pod	logs of the wrong container	<code>-c NAME</code> or <code>--all-containers --prefix</code>
Deleting a broken Pod "to see if it helps"	the evidence (logs, events) is gone, the new Pod fails the same way	read events and logs first, then fix the cause
Expecting a shell in every image	<code>executable file not found</code>	<code>kubectl debug</code> with a tools image
Searching the error text first	other people's problems	follow the method: object → events → logs → configuration

KEY TAKEAWAYS

- Method: what is broken → expected vs actual → which object → describe/events → logs → configuration → fix → verify.
- Events (`kubectl get events --sort-by=.lastTimestamp` , `describe`) explain most problems at once.
- `kubectl logs --previous` for crashes, `-c` for multi-container Pods; exit codes tell how a container ended.
- `kubectl exec` runs tools that exist in the image; `kubectl debug --target` brings your own.
- Real-world use: on-call engineers follow exactly this order on production clusters, often under time pressure; the method keeps them from guessing.

Full lesson with every command and its real output: [27-troubleshooting](#)

28 · Helm

Helm is the package manager of Kubernetes. A **chart** is a folder of YAML templates plus a file of default settings (`values.yaml`); Helm fills the templates with the values and applies the result to the cluster as one unit, a **release**. Every change to a release is numbered (a **revision**), so you can see the history and roll back.

If raw YAML files are a recipe written out for exactly four people, a chart is the recipe with blanks: "flour: {{ guests × 100 }} g". You write it once and cook it for 2, 4 or 40 people by changing one number.

WHY

By now the course application is a dozen YAML files: Deployments, Services, a ConfigMap, a Secret, a PVC. Running it in **dev** and **test** means copying all of them and changing a few values in each copy, then keeping the copies in sync forever. Helm replaces the copies with **one chart and one small values file per environment**, installs and upgrades everything in one command, and remembers each version for rollbacks.

```
Raw YAML          → many files, copied per environment, edited by hand
Helm chart + values → one set of templates; dev, test, prod differ only in a values file
helm upgrade / rollback → versioned releases of the whole application
```

HOW IT WORKS

```
helm/learning-app/          values-dev.yaml      values-test.yaml
├─ Chart.yaml               (name, version) backend.replicas: 1  backend.replicas: 2
├─ values.yaml              (defaults)      message: DEV          database.enabled: true
└─ templates/               (YAML with {{ }})
    │
    ▼ helm install shop helm/learning-app -f values-dev.yaml
Helm renders the templates with: values.yaml, then the -f file, then --set (later wins)
    │
    ▼
plain Kubernetes YAML → API server (stored as release "shop", revision 1, in a Secret in the namespace)
```

Word	Meaning
chart	the package: templates + default values + metadata
values	the settings that fill the templates (<code>values.yaml</code> , <code>-f file</code> , <code>--set key=value</code>)
release	one installation of a chart, with a name (<code>shop</code>), in a namespace
revision	one version of a release: install = 1, each upgrade or rollback adds one

ARCHITECTURE

```
values-dev.yaml ─┐
                  │
values-test.yaml ─┴─ chart helm/learning-app
                    templates + values.yaml
                    ──→ release "shop" (namespace helm-lab)
                        revision 1: backend 1.0.0, 1 replica
                        revision 2: backend 2.0.0           ← helm upgrade
                        revision 3: = revision 1             ← helm rollback 1
```

COMMON MISTAKES

Mistake	Symptom	Fix
Editing objects with <code>kubectl</code> after <code>helm install</code>	the next <code>helm upgrade</code> overwrites or conflicts with your change	change values and <code>helm upgrade</code> ; the chart is the source of truth
Forgetting <code>-f values-dev.yaml</code> on upgrade	the release silently goes back to the defaults	pass the same values files every time (or <code>--reuse-values</code> , carefully)

Mistake	Symptom	Fix
<code>helm install</code> twice with the same name	cannot re-use a name that is still in use	<code>helm upgrade --install</code> installs or upgrades
No <code>--wait</code>	STATUS: deployed although the Pods crash	<code>--wait --timeout</code> makes Helm check readiness
Indentation errors in templates	error converting YAML to JSON	<code>helm template</code> and <code>helm lint</code> before every install

KEY TAKEAWAYS

- A chart = templates + default values; a release = one installation; every change is a numbered revision.
- `helm install / upgrade / rollback / uninstall / list / history`; `lint` and `template` before installing.
- Values files per environment replace copies of YAML: dev, test and prod differ only in their values.
- `--wait --timeout` makes failures visible; `helm rollback` returns to any earlier revision.
- Real-world use: most third-party software (ingress controllers, databases, monitoring) is installed with Helm, and many teams package their own applications as charts that CI/CD pipelines upgrade.

Full lesson with every command and its real output: [28-helm](#)

Level 16 · The capstone

29 · Capstone: a production-style application

The whole course in one application. Everything you learned, used once, on purpose: a frontend, a backend API and a PostgreSQL database in their own namespace, reachable through an Ingress, configured with ConfigMaps and a Secret, persistent, health-checked, limited, autoscaled, backed up every night, isolated by NetworkPolicies and protected by RBAC. You deploy it, inspect it, access it, scale it, update it, roll it back, and then break it six ways and fix it.

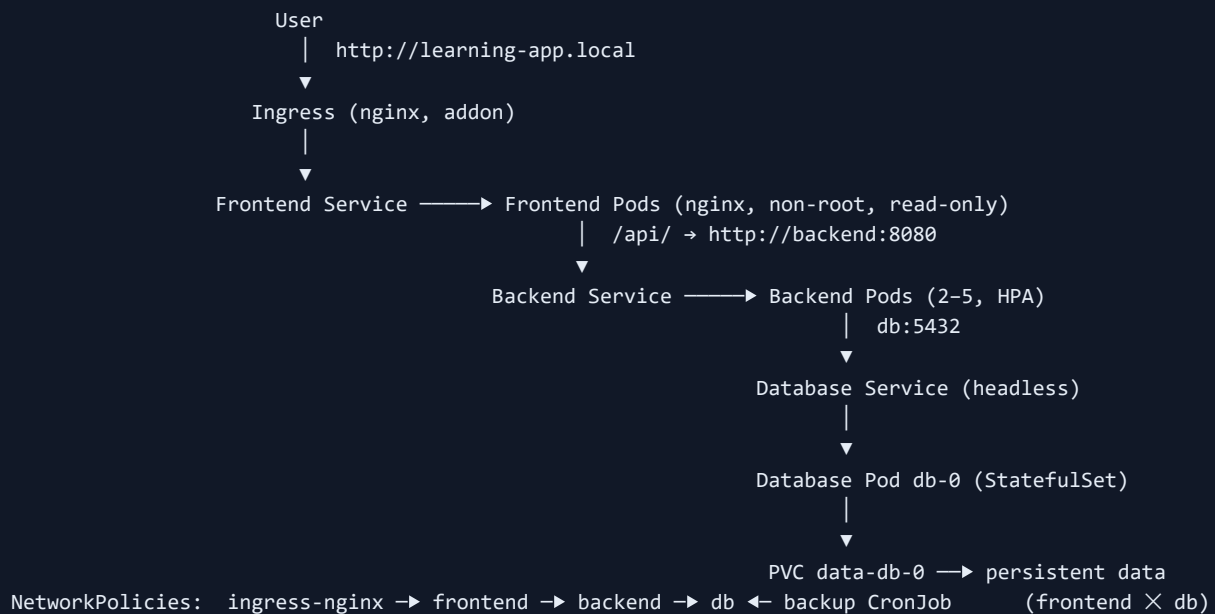
WHY

Each lesson taught one object in isolation. Real applications are made of many objects that depend on each other: a Service that selects the wrong label, a ConfigMap with one wrong value or a NetworkPolicy that is too strict breaks the whole chain, and the symptom appears somewhere else. The capstone trains exactly that: following a symptom through the objects to its cause.

HOW IT WORKS

Concept	Where in the capstone	Lesson
Namespace	<code>learning-app</code>	05
Deployments, ReplicaSets, Pods, labels, selectors	frontend and backend	06–09
Services, DNS	<code>frontend</code> , <code>backend</code> , headless <code>db</code> ; the frontend proxies to <code>http://backend:8080</code>	10
ConfigMaps	backend settings (env), frontend <code>nginx.conf</code> and page (files)	11
Secret	database credentials, the password mounted as a file	12
StatefulSet, PVC, StorageClass	PostgreSQL with a 1 Gi volume from <code>standard</code> ; backups on their own PVC	13, 19
Probes	startup, liveness and readiness on every container	14
Requests and limits	every container	15
CronJob, Job	nightly <code>pg_dump</code> , started by hand with <code>kubect1 create job --from=cronjob/...</code>	17
Ingress	<code>learning-app.local</code> → frontend	20
NetworkPolicies	default deny; ingress → frontend → backend → database ← backup	22
ServiceAccounts	one per component, no API token mounted	23
RBAC	the user <code>developer</code> may view, not change	24
Rolling updates, rollbacks, scaling, HPA	backend 1.0.0 → 2.0.0 → 1.0.0; HPA 2–5 Pods	09, 25, 26
Helm	the same application from the course chart	28

ARCHITECTURE



COMMON MISTAKES

Mistake	Symptom	Fix
Applying everything at once into a missing namespace	namespaces "learning-app" not found	the namespace first, then the rest
Fixing the live object by hand (kubectl edit)	the next <code>apply</code> brings the bug back, or undoes the fix	fix the file, apply the file
Reading only one Pod's state	the cause is in another object (Service, ConfigMap, policy)	follow the chain: Ingress → Service → endpoints → Pods → config → dependencies
Removing a NetworkPolicy to "make it work"	the security goal is lost	correct the selector; test the allowed and the forbidden path

KEY TAKEAWAYS

- A working application is a chain of objects; a break anywhere shows up at the user.
- Troubleshoot from the outside in: user → Ingress → Service → endpoints → Pods → events → logs → configuration → dependencies → policies.
- Fix the reviewed files and apply them; verify as the user, then check the parts the user does not see (backups).
- Real-world use: this is the shape of most applications on Kubernetes, and the six breaks are the six most common causes of incidents after a change.

Full lesson with every command and its real output: [29-capstone](#)

Lab 12 · Troubleshooting

Level 14 · Troubleshooting · ⌚ 2–3 hours · run every command from the course folder · cluster: minikube

Fourteen broken setups, each one a problem you will meet on a real cluster. Every problem file creates the broken setup in its own namespace (`trouble-NN`), shows the real symptoms, and then works through:

Problem → Symptoms → First command to run → Investigation → Root cause → Fix → Verification → Lesson learned

Try to find the root cause **yourself** after the "First command to run", before you read the Investigation. The method behind every investigation is in [lesson 27](#):

What is broken? → What should happen? → What actually happened? → Which object controls it?
→ inspect it → events → logs → configuration → root cause → fix → verify

Problems by symptom

#	Symptom you see	Problem
01	Pending , NODE <none>	Pod stuck in Pending
02	CrashLoopBackOff , restarts grow	CrashLoopBackOff
03	ErrImagePull , ImagePullBackOff	ImagePullBackOff
04	Completed / CrashLoopBackOff , empty logs, exit code 0	Container exits immediately
05	Service: Connection refused , Endpoints: <none>	Service cannot reach Pods
06	Running but READY 0/1	Readiness probe failing
07	CreateContainerConfigError: couldn't find key	Wrong ConfigMap
08	CreateContainerConfigError: secret not found	Wrong Secret
09	PVC and Pod Pending	PVC stuck in Pending
10	Ingress answers 503	Ingress not working
11	rollout status never finishes	Deployment rollout stuck
12	OOMKilled , exit code 137, logs stop	Pod OOMKilled
13	bad address	DNS resolution failure
14	a Service with endpoints times out	NetworkPolicy blocks traffic

The broken manifests are in [manifests/](#). More practice without the answers next to the problem: [troubleshooting challenges](#).

Problem 01 · Pod stuck in Pending

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

A colleague deployed the nightly report service. Hours later it still has not run once. "The Pod is just waiting", they say. Set up their Deployment:

```
kubectl create namespace trouble-01
kubectl apply -f labs/12-troubleshooting/manifests/01-pending.yaml -n trouble-01
sleep 5
```

Symptoms

```
kubectl get pods -n trouble-01 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
report-bb7b9fdb9-z6p1b	0/1	Pending	0	5s	<none>	<none>	<none>	<none>

Pending and NODE <none>: the Pod has not been placed on any node yet.

First command to run

The Pod's events: the scheduler writes there why it could not place a Pod.

```
kubectl describe pod -n trouble-01 -l app=report | grep -A4 '^Events'
```

```
Events:
  Type      Reason           Age   From                  Message
  ----      -
Warning    FailedScheduling 5s    default-scheduler    0/1 nodes are available: 1 Insufficient cpu.
preemption: 0/1 nodes are available: 1 Preemption is not helpful for scheduling.
```

Investigation

What is broken? The Pod is never scheduled. *Which object controls this?* The scheduler, using the Pod's resource **requests** and what each node can still offer. The event names the reason: `Insufficient cpu`. Compare what the Pod asks for with what the node has:

```
kubectl get pods -n trouble-01 -l app=report -o jsonpath='requests:
{.items[0].spec.containers[0].resources.requests}{"\n"}'
kubectl get node minikube -o jsonpath='allocatable cpu: {status.allocatable.cpu}{"\n"}'
```

```
requests: {"cpu":"64","memory":"32Mi"}
allocatable cpu: 14
```

Root cause

The Pod requests `cpu: "64"`, 64 whole CPUs (the author meant `64m`, 64 millicores). No node can offer that, so the scheduler leaves the Pod `Pending` forever. Nothing is "slow": it will never start.

Fix

Request what the application really needs. Fix the manifest and apply it; the Deployment replaces the Pod:

```
sed 's/cpu: "64"          # 64 whole CPUs/cpu: 100m/' labs/12-troubleshooting/manifests/01-pending.yaml |
kubectl apply -n trouble-01 -f -
kubectl rollout status deployment/report -n trouble-01 --timeout=120s
```

Verification

```
kubectl get pods -n trouble-01 -o wide
```

The Pod is `Running` on `minikube`.

Lesson learned

- `Pending` + `NODE <none>` = a scheduling problem: read the `FailedScheduling` event first.
- The scheduler plans with **requests**, not with real usage: an impossible request blocks a Pod even on an idle node.
- `64` is 64 CPUs; `64m` is 0.064 of a CPU. Units matter (lesson 15).

Cleanup

✂ Delete the namespace `trouble-01` and the report Deployment in it:

```
kubectl delete namespace trouble-01
```

Problem 02 · CrashLoopBackOff

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The new queue worker was deployed this morning. Its Pod shows a restart count that keeps growing. Set it up:

```
kubectl create namespace trouble-02
kubectl apply -f labs/12-troubleshooting/manifests/02-crashloop.yaml -n trouble-02
```

Symptoms

```
kubectl get pods -n trouble-02 | grep CrashLoopBackOff
```

```
worker-5b6dd64668-nkhvt    0/1    CrashLoopBackOff    1 (2s ago)    4s
```

CrashLoopBackOff: the container starts, exits, is restarted, exits again; Kubernetes waits longer and longer between attempts (10 s, 20 s, 40 s... up to 5 minutes).

First command to run

The logs of the container that crashed. The current container may not have started yet, so ask for the **previous** one:

```
kubectl logs -n trouble-02 -l app=worker --previous
```

```
fatal: QUEUE_URL is not set
worker starting
```

Investigation

What should happen? A long-running worker. *What happened?* It exits during start-up. The exit code confirms that the application itself chose to stop (code 1), as opposed to being killed (137, problem 12):

```
kubectl get pods -n trouble-02 -l app=worker -o jsonpath='{last state:
{.items[0].status.containerStatuses[0].lastState.terminated.reason}, exit code
{.items[0].status.containerStatuses[0].lastState.terminated.exitCode}}{"\n"}'
```

```
last state: Error, exit code 1
```

Which environment variables does the container get?

```
kubectl get deployment worker -n trouble-02 -o jsonpath='{.spec.template.spec.containers[0].env}{"\n"}'
```

None at all.

Root cause

The worker needs `QUEUE_URL` and refuses to start without it; the Deployment does not set it. Kubernetes restarts the container exactly as configured, so it fails the same way every time.

Fix

Give the container its configuration (in real life from a ConfigMap, lesson 11):

```
kubectl set env deployment/worker -n trouble-02 QUEUE_URL=amqp://queue:5672
kubectl rollout status deployment/worker -n trouble-02 --timeout=120s
```

Verification

```
kubectl logs -n trouble-02 -l app=worker
```

Lesson learned

- `CrashLoopBackOff` is not the error; it is Kubernetes waiting between restarts. The error is in the logs.
- `kubectl logs --previous` shows the container that crashed; without it you may see an empty, just-started one.
- Exit code 1 (or any small number) = the application stopped itself; 137 = killed (out of memory, problem 12).

Cleanup

✂ Delete the namespace `trouble-02` and the worker in it:

```
kubectl delete namespace trouble-02
```

Problem 03 • ImagePullBackOff

Lab 12 • Troubleshooting • ⌚ 10 minutes • run every command from the course folder • cluster: minikube

Problem

A web server was deployed "with the usual nginx image", but it never starts. Set it up:

```
kubectl create namespace trouble-03
kubectl apply -f labs/12-troubleshooting/manifests/03-imagepull.yaml -n trouble-03
```

Symptoms

```
kubectl get pods -n trouble-03 | grep -E 'ImagePullBackOff|ErrImagePull'
```

```
web-597f85f646-26k2z    0/1    ErrImagePull    0    2s
```

`ErrImagePull` (the attempt failed) then `ImagePullBackOff` (Kubernetes waits before trying again).

First command to run

The events say which image was requested and what the registry answered:

```
kubectl describe pod -n trouble-03 -l app=web | grep -m1 'Failed to pull image'
```

```
Warning   Failed      1s    kubelet                spec.containers{web}: Failed to pull image "nginx:1.30-alpne":
rpc error: code = NotFound desc = failed to pull and unpack image "docker.io/library/nginx:1.30-alpne": failed
to resolve reference "docker.io/library/nginx:1.30-alpne": docker.io/library/nginx:1.30-alpne: not found
```

Investigation

Which object is involved? The kubelet tried to download the image named in the Pod spec, from the registry (`docker.io/library/nginx`), and the registry said `not found`. Read the name carefully: `1.30-alpne`. Compare it with a tag you know works:

```
kubectl get deployment web -n trouble-03 -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Other answers mean other causes: `pull access denied` / `unauthorized` (a private image without credentials, `imagePullSecrets`), `429 Too Many Requests` (Docker Hub's rate limit), a timeout (no network from the node).

Root cause

A typo in the tag: `alpne` instead of `alpine`. No such image exists, so the kubelet can never start the container.

Fix

```
kubectl set image deployment/web -n trouble-03 web=nginx:1.30-alpine
kubectl rollout status deployment/web -n trouble-03 --timeout=120s
```

Verification

```
kubectl get pods -n trouble-03
```

Lesson learned

- `ErrImagePull` / `ImagePullBackOff`: read the `Failed to pull image` event, it quotes the exact name and the registry's answer.
- `not found` = wrong name or tag; `unauthorized` = credentials; `429` = rate limit; `timeout` = network.
- Copy image references from a source that was tested; pin exact tags.

Cleanup

✂ Delete the namespace `trouble-03` and the web Deployment in it:

```
kubectl delete namespace trouble-03
```


Problem 04 · Container exits immediately

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The team wants a "toolbox" Pod with BusyBox to run debugging commands in. It keeps restarting, although nothing is wrong in its logs. Set it up:

```
kubectl create namespace trouble-04
kubectl apply -f labs/12-troubleshooting/manifests/04-exits-immediately.yaml -n trouble-04
```

Symptoms

```
kubectl get pods -n trouble-04 | grep -E 'Completed|CrashLoopBackOff'
```

```
toolbox-65989c87c-xzkwn    0/1    Completed    0    2s
```

First command to run

The logs first, as for any crash:

```
kubectl logs -n trouble-04 -l app=toolbox --previous 2>&1 | head -3
```

Empty. Nothing was printed, so look at how the container ended:

```
kubectl get pods -n trouble-04 -l app=toolbox -o jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated.reason}, exit code {.items[0].status.containerStatuses[0].lastState.terminated.exitCode}{"\n"}'
```

```
last state: Completed, exit code 0
```

Investigation

Exit code 0, reason `Completed`: the container did not crash, it **finished successfully**. A container lives exactly as long as its main process. Which process is that?

```
kubectl get deployment toolbox -n trouble-04 -o jsonpath='{.spec.template.spec.containers[0].command} {"\n"}'
minikube image ls | grep busybox | head -1
```

No `command` in the Deployment, so the image's default runs: BusyBox's default command is `sh`. A shell without a terminal (no `-it`) has nothing to read, so it exits at once with 0. A Deployment exists to keep a process **running**, so it restarts the container, which exits again: `CrashLoopBackOff` with exit code 0.

Root cause

The container has no long-running process. Deployments are for programs that run until stopped; a command that ends belongs in a **Job** (lesson 17).

Fix

Give the toolbox something that keeps running (here `sleep`), so the team can `kubectl exec` into it:

```
kubectl patch deployment toolbox -n trouble-04 --type=json \
  -p '[{"op":"add","path":"/spec/template/spec/containers/0/command","value":["sleep","infinity"]}]'
kubectl rollout status deployment/toolbox -n trouble-04 --timeout=120s
```

Verification

```
kubectl get pods -n trouble-04
kubectl exec -n trouble-04 deploy/toolbox -- busybox | head -1
```

Lesson learned

- A container runs as long as its main process; a Deployment restarts anything that ends, even successfully.
- `CrashLoopBackOff` with **exit code 0** and empty logs = the process simply finished: wrong command, or the work belongs in a Job.
- Check the image's default command (`ENTRYPOINT` / `CMD`) when a Pod spec sets none.

Cleanup

 Delete the namespace `trouble-04` and the toolbox in it:

```
kubectl delete namespace trouble-04
```

Problem 05 · Service cannot reach Pods

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The backend Pods run fine, but every request to the `backend` Service fails. Set it up:

```
kubectl create namespace trouble-05
kubectl apply -f labs/12-troubleshooting/manifests/05-service-no-endpoints.yaml -n trouble-05
kubectl rollout status deployment/backend -n trouble-05 --timeout=120s
```

Symptoms

```
kubectl run client -n trouble-05 --restart=Never --image=busybox:1.37 -- wget -qO- -T 5 http://backend > /dev/null
kubectl wait --for=jsonpath='{.status.phase}'=Failed pod/client -n trouble-05 --timeout=60s > /dev/null
kubectl logs client -n trouble-05
```

```
wget: can't connect to remote host (10.111.243.195): Connection refused
```

The name `backend` resolved to an address (DNS works), but nothing accepted the connection.

First command to run

Does the Service have **endpoints**, the Pod addresses it forwards to?

```
kubectl describe service backend -n trouble-05 | grep -E '^(Selector|Endpoints)'
```

```
Selector:          app=back-end
Endpoints:
```

Investigation

Which object decides where a Service sends traffic? Its **selector**: every ready Pod whose labels match it becomes an endpoint. No endpoints means no Pod matches. Compare the selector with the Pods' labels:

```
kubectl get pods -n trouble-05 --show-labels
kubectl get pods -n trouble-05 -l app=back-end 2>&1
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
backend-5599cd7d88-5jt79	1/1	Running	0	5s	app=backend,pod-template-hash=5599cd7d88
backend-5599cd7d88-868z4	1/1	Running	0	5s	app=backend,pod-template-hash=5599cd7d88
client	0/1	Error	0	4s	run=client

No resources found in trouble-05 namespace.

The Pods say `app=backend`; the Service looks for `app=back-end`.

Root cause

A typo in the Service selector. Labels match exactly; one character is enough to match nothing (lesson 07).

Fix

```
kubectl patch service backend -n trouble-05 -p '{"spec":{"selector":{"app":"backend"}}}'
```

Verification

```
kubectl describe service backend -n trouble-05 | grep '^Endpoints'
kubectl run client2 -n trouble-05 --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
  sh -c 'for i in $(seq 1 10); do wget -qO- -T 5 http://backend && exit 0; sleep 2; done; exit 1'
```

Lesson learned

- "Service does not work" → first `kubectl describe service`: an empty `Endpoints:` points at the selector or at Pods that are not ready (problem 06).
- `kubectl get pods -l SELECTOR` with the Service's exact selector shows what it matches.
- `Connection refused` from a Service IP = no endpoints; a timeout = endpoints exist but do not answer (or a NetworkPolicy, problem 14).

Cleanup

 Delete the namespace `trouble-05` (backend Deployment, Service, client Pod):

```
kubectl delete namespace trouble-05
```

Problem 06 · Readiness probe failing

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

After someone "added health checks", the backend Pods run but the Service sends them nothing. Set it up:

```
kubectl create namespace trouble-06
kubectl apply -f labs/12-troubleshooting/manifests/06-readiness.yaml -n trouble-06
sleep 15
```

Symptoms

```
kubectl get pods -n trouble-06
```

NAME	READY	STATUS	RESTARTS	AGE
backend-6b89dbb776-2mxr7	0/1	Running	0	15s
backend-6b89dbb776-kkmr4	0/1	Running	0	15s

Running but READY 0/1 : the container runs, Kubernetes does not consider it ready for traffic. The events say why (`kubectl describe pod` shows the same events at the end of its output):

First command to run

```
kubectl get events -n trouble-06 --field-selector reason=Unhealthy -o custom-columns=MESSAGE:.message | grep -m1 'Readiness probe failed'
```

```
Readiness probe failed: HTTP probe failed with statuscode: 404
```

Investigation

statuscode: 404 : the kubelet reached the application (so the port is right), but the **path** does not exist. Which path does the probe ask, and which paths does the application have?

```
kubectl get deployment backend -n trouble-06 -o jsonpath='{.spec.template.spec.containers[0].readinessProbe.httpGet.path}'
```

The backend serves `/readyz` and `/livez` (lesson 14), not `/ready`. Because no Pod is ready, the Service has no endpoints:

```
kubectl describe service backend -n trouble-06 | grep '^Endpoints'
```

Root cause

The readiness probe checks `/ready`, a path the application does not have. Every check gets 404, so the Pods never become ready and receive no traffic. (A failing **liveness** probe would also restart them; readiness only removes them from the Service.)

Fix

```
kubectl patch deployment backend -n trouble-06 --type=json \
-p
'[{ "op": "replace", "path": "/spec/template/spec/containers/0/readinessProbe/httpGet/path", "value": "/readyz" } ]'
kubectl rollout status deployment/backend -n trouble-06 --timeout=120s
```

Verification

```
kubectl get deployment backend -n trouble-06
```

Lesson learned

- Running + 0/1 READY = a readiness problem: the Unhealthy event says why.
- The HTTP status tells you where to look: 404 path, 500/503 the application, connection refused the port.
- Test a probe's URL by hand before you put it in a manifest.

Cleanup

✂ Delete the namespace trouble-06 :

```
kubectl delete namespace trouble-06
```

Problem 07 · Wrong ConfigMap

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The backend's configuration moved into a ConfigMap. Since then its Pod never starts. Set it up:

```
kubectl create namespace trouble-07
kubectl apply -f labs/12-troubleshooting/manifests/07-configmap.yaml -n trouble-07
```

Symptoms

```
kubectl get pods -n trouble-07 | grep CreateContainerConfigError
```

```
backend-69d5b6f5d4-2tz86    0/1    CreateContainerConfigError    0    2s
```

`CreateContainerConfigError`: the image is there, but the kubelet cannot even create the container, because part of its configuration is missing.

First command to run

```
kubectl describe pod -n trouble-07 -l app=backend | grep -m1 'Error:'
```

```
Warning   Failed    0s (x3 over 2s)  kubelet                spec.containers{backend}: Error: couldn't find key
log_level in ConfigMap trouble-07/backend-config
```

Investigation

The ConfigMap exists, but the key the Deployment asks for does not. Which keys does it have?

```
kubectl get configmap backend-config -n trouble-07 -o jsonpath='{.data}{"\n"}'
```

```
{"LOG_LEVEL":"info","MESSAGE":"Hello from the ConfigMap"}
```

Keys are case-sensitive: the ConfigMap has `LOG_LEVEL`, the Deployment asks for `log_level`.

Root cause

The Deployment refers to a ConfigMap key that does not exist. A missing key (or ConfigMap) blocks the container from starting, unless the reference is marked `optional: true`.

Fix

Point the reference at the real key:

```
kubectl patch deployment backend -n trouble-07 --type=json \
-p
' [{"op": "replace", "path": "/spec/template/spec/containers/0/env/1/valueFrom/configMapKeyRef/key", "value": "LOG_LEVEL"} ]'
kubectl rollout status deployment/backend -n trouble-07 --timeout=120s
```

Verification

The backend shows the configuration it received:

```
ip=$(kubectl get pod -n trouble-07 -l app=backend -o jsonpath='{.items[0].status.podIP}')
kubectl run client -n trouble-07 --rm -i --quiet --restart=Never --image=busybox:1.37 -- wget -qO-
"http://$ip:8080/api/config"
```

Lesson learned

- `CreateContainerConfigError` = a ConfigMap, Secret or key the Pod needs is missing: the event names it exactly.
- ConfigMap keys are case-sensitive; `kubectl get configmap NAME -o jsonpath='{.data}'` lists them.
- References are checked when the container starts, not when you apply the Deployment.

Cleanup

✂ Delete the namespace `trouble-07`:

```
kubectl delete namespace trouble-07
```


Problem 08 · Wrong Secret

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The database password was put into a Secret, and the backend was told to read it. The backend never starts. Set it up:

```
kubectl create namespace trouble-08
kubectl apply -f labs/12-troubleshooting/manifests/08-secret.yaml -n trouble-08
```

Symptoms

```
kubectl get pods -n trouble-08 | grep CreateContainerConfigError
```

backend-7864884799-64qqz	0/1	CreateContainerConfigError	0	3s
--------------------------	-----	----------------------------	---	----

First command to run

```
kubectl describe pod -n trouble-08 -l app=backend | grep -m1 'Error:'
```

Warning	Failed	1s (x3 over 2s)	kubelet	spec.containers{backend}: Error: secret "db-secret" not found
---------	--------	-----------------	---------	---

Investigation

Which Secrets exist in the namespace? (Listing Secrets shows their names, never their values.)

```
kubectl get secrets -n trouble-08
```

NAME	TYPE	DATA	AGE
db-credentials	Opaque	1	3s

The Secret is called `db-credentials`; the Deployment asks for `db-secret`. Secrets are also namespaced: a Secret with the right name in another namespace would not help either.

Root cause

The Deployment references a Secret name that does not exist in its namespace, so the kubelet cannot build the container's environment.

Fix

```
kubectl patch deployment backend -n trouble-08 --type=json \
-p
' [{"op": "replace", "path": "/spec/template/spec/containers/0/env/0/valueFrom/secretKeyRef/name", "value": "db-credentials"} ]'
kubectl rollout status deployment/backend -n trouble-08 --timeout=120s
```

Verification

The backend reports that it received a password, without showing it:

```
ip=$(kubectl get pod -n trouble-08 -l app=backend -o jsonpath='{.items[0].status.podIP}')
kubectl run client -n trouble-08 --rm -i --quiet --restart=Never --image=busybox:1.37 -- wget -qO-
"http://$ip:8080/api/config"
```

Lesson learned

- `secret "X" not found` = a wrong name or the wrong namespace; `couldn't find key` = a wrong key (problem 07).
- `kubectl get secrets -n NS` lists names safely; never print a Secret's values while troubleshooting.
- Keep Secret names in one place (a Helm value, lesson 28) so the Secret and its users cannot drift apart.

Cleanup

✂ Delete the namespace `trouble-08` (Deployment and Secret):

```
kubectl delete namespace trouble-08
```

Problem 09 · PVC stuck in Pending

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

A Pod that writes data to a volume never starts. Set it up:

```
kubectl create namespace trouble-09
kubectl apply -f labs/12-troubleshooting/manifests/09-pvc-pending.yaml -n trouble-09
sleep 5
```

Symptoms

```
kubectl get pod,pvc -n trouble-09
```

NAME	READY	STATUS	RESTARTS	AGE
pod/writer	0/1	Pending	0	5s

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS
AGE						
persistentvolumeclaim/data	Pending				fast-ssd	<unset>
5s						

Both are **Pending**: the Pod waits for its volume, the claim waits for storage.

First command to run

Start with the object at the bottom of the chain, the claim:

```
kubectl describe pvc data -n trouble-09 | grep -A3 '^Events'
```

```
Events:
  Type      Reason             Age   From                      Message
  ----      -
  Warning   ProvisioningFailed  5s    persistentvolume-controller storageclass.storage.k8s.io "fast-ssd" not found
```

Investigation

Which StorageClasses does the cluster have?

```
kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
standard (default)	k8s.io/minikube-hostpath	Delete	Immediate	false	13h

Only **standard**. The claim asks for **fast-ssd**, a class from another cluster (a cloud provider's, perhaps), copied into this manifest.

Root cause

The PVC names a StorageClass that does not exist here, so no provisioner creates a volume; the Pod waits for the claim forever. (Other causes of a `Pending` PVC: no default StorageClass, or a class with `WaitForFirstConsumer` that waits until a Pod uses it.)

Fix

A PVC's `storageClassName` cannot be changed, so recreate the claim without it (the default class is used):

```
kubect1 delete -n trouble-09 pod/writer pvc/data
sed '/storageClassName: fast-ssd/d' labs/12-troubleshooting/manifests/09-pvc-pending.yaml | kubect1 apply -n trouble-09 -f -
kubect1 wait --for=condition=Ready pod/writer -n trouble-09 --timeout=120s > /dev/null
kubect1 get pod writer -n trouble-09
```

Verification

```
kubect1 get pvc data -n trouble-09
kubect1 exec writer -n trouble-09 -- cat /data/started.txt
```

Lesson learned

- A Pod `Pending` because of a volume: look at the PVC, then its events.
- `kubect1 get storageclass` shows what this cluster can provision; class names differ between clusters.
- PVC specs are mostly immutable: fix the manifest and recreate the claim (data in a bound volume needs a plan first).

Cleanup

✂ Delete the namespace `trouble-09` (the Pod, the claim and, through the `Delete` reclaim policy, its volume):

```
kubect1 delete namespace trouble-09
```

Problem 10 • Ingress not working

Lab 12 • Troubleshooting • ⌚ 15 minutes • run every command from the course folder • cluster: minikube

Problem

The shop's website should answer on `shop.local` through the Ingress. The web Pod runs, the Service exists, but the site answers with an error. Set it up (the Ingress controller is Minikube's `ingress` addon, lesson 20):

```
minikube addons enable ingress 2>&1 | tail -1
kubectl wait --for=condition=Ready pod -n ingress-nginx -l app.kubernetes.io/component=controller --
timeout=300s > /dev/null
kubectl create namespace trouble-10
kubectl apply -f labs/12-troubleshooting/manifests/10-ingress.yaml -n trouble-10
kubectl rollout status deployment/web -n trouble-10 --timeout=120s
```

Symptoms

Call the Ingress controller with the host name, from a client inside the cluster:

```
kubectl run client -n trouble-10 --rm -i --quiet --restart=Never --image=busybox:1.37 -- sh -c '
for i in $(seq 1 15); do
  out=$(wget -qO- -T 5 --header "Host: shop.local" http://ingress-nginx-controller.ingress-nginx 2>&1)
  case "$out" in *503*) break;; esac; sleep 2
done; echo "$out" | tail -1'
```

```
wget: server returned error: HTTP/1.1 503 Service Temporarily Unavailable
```

`503` from the controller: it received the request and recognised the host, but has nowhere to send it.

First command to run

```
kubectl describe ingress shop -n trouble-10 | grep -A4 '^Rules'
```

```
Rules:
  Host      Path  Backends
  ----      -
  shop.local
           /    frontend:80 (<error: services "frontend" not found>)
```

Investigation

The rule sends `/` to a Service called `frontend`, and Kubernetes says it does not exist. Which Services exist?

```
kubectl get services -n trouble-10
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
web	ClusterIP	10.99.0.235	<none>	80/TCP	6s

Other things to check when an Ingress fails: the `ingressClassName` (no controller picks up the Ingress → 404 from the default backend or no answer), the host header (another host → 404), the Service port number, and whether

the Service has endpoints (problem 05).

Root cause

The Ingress points at a Service name (`frontend`) that does not exist in its namespace; the Service is called `web` .

Fix

```
kubectl patch ingress shop -n trouble-10 --type=json \
-p '[{"op":"replace","path":"/spec/rules/0/http/paths/0/backend/service/name","value":"web"}]'
```

Verification

```
kubectl run client2 -n trouble-10 --rm -i --quiet --restart=Never --image=busybox:1.37 -- sh -c '
for i in $(seq 1 15); do
  wget -qO- -T 5 --header "Host: shop.local" http://ingress-nginx-controller.ingress-nginx && exit 0; sleep
2
done; exit 1' | grep -o '<title>.*</title>'
```

The controller retried in the loop above because it picks up Ingress changes within a few seconds, not instantly.

Lesson learned

- Ingress problems are a chain: controller → rule (host, path) → Service → endpoints → Pods. Check each link.
- `kubectl describe ingress` shows each backend and an `<error: ...>` when the Service is missing.
- 503 = the rule matched but the backend is missing or has no endpoints; 404 = no rule matched (host or path).

Cleanup

✂ Delete the namespace `trouble-10` (Deployment, Service, Ingress). The ingress addon stays enabled for later lessons.

```
kubectl delete namespace trouble-10
```

Problem 11 · Deployment rollout stuck

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

Version 3.0.0 of the backend was released this afternoon. The deploy step has been "waiting for rollout" ever since. Set up the running version, then the release:

```
kubectl create namespace trouble-11
kubectl apply -f labs/12-troubleshooting/manifests/11-rollout.yaml -n trouble-11
kubectl rollout status deployment/backend -n trouble-11 --timeout=120s
kubectl set image deployment/backend -n trouble-11 backend=learning-app/backend:3.0.0
```

Symptoms

```
kubectl rollout status deployment/backend -n trouble-11 --timeout=30s 2>&1
```

```
Waiting for deployment "backend" rollout to finish: 1 out of 3 new replicas have been updated...
error: timed out waiting for the condition
```

First command to run

```
kubectl get pods -n trouble-11 -o custom-
columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image,READY:.status.containerStatuses[0].ready,REASON:.s
tatus.containerStatuses[0].state.waiting.reason
```

NAME	IMAGE	READY	REASON
backend-6fc7cbfff9-f7z2f	learning-app/backend:1.0.0	true	<none>
backend-6fc7cbfff9-hdxvk	learning-app/backend:1.0.0	true	<none>
backend-6fc7cbfff9-vqbfq	learning-app/backend:1.0.0	true	<none>
backend-7c96cd4d65-9gzpz	learning-app/backend:3.0.0	false	ImagePullBackOff

Investigation

The rolling update started one Pod with the new version, and that Pod cannot pull its image. The Deployment waits for it to become ready before replacing more old Pods, so the three old Pods keep serving: **no outage**, just a rollout that never finishes. Why can the new image not be pulled?

```
kubectl get events -n trouble-11 --field-selector reason=Failed -o custom-columns=MESSAGE:.message | grep -m1
'3.0.0'
minikube image ls | grep learning-app/backend
```

The cluster only has 1.0.0 and 2.0.0; there is no 3.0.0 locally or in any registry.

Root cause

The release references an image that was never built. The rolling update protects the users (old Pods stay), but the rollout stays stuck until someone acts.

Fix

Go back to the last working version (lesson 09), then deal with the release:

```
kubectl rollout undo deployment/backend -n trouble-11
kubectl rollout status deployment/backend -n trouble-11 --timeout=120s
```

Verification

```
kubectl get pods -n trouble-11 -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image
kubectl rollout history deployment/backend -n trouble-11
```

Lesson learned

- A stuck rollout = a new Pod that never becomes ready; look at the **new** Pods, not the old ones.
- Rolling updates keep the old version serving while the new one fails: users are fine, the deploy is not.
- `kubectl rollout undo` is the fast, safe way back; set `progressDeadlineSeconds` so a stuck rollout is reported.

Cleanup

✂ Delete the namespace `trouble-11`:

```
kubectl delete namespace trouble-11
```


Problem 12 · Pod OOMKilled

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The nightly import Pod stops in the middle of its work. Its logs end without any error. Set it up:

```
kubectl create namespace trouble-12
kubectl apply -f labs/12-troubleshooting/manifests/12-oomkilled.yaml -n trouble-12
```

Symptoms

```
kubectl get pod import -n trouble-12
kubectl get pod import -n trouble-12 | grep -q OOMKilled
```

NAME	READY	STATUS	RESTARTS	AGE
import	0/1	OOMKilled	0	5s

First command to run

```
kubectl logs import -n trouble-12
```

```
loading 100 MiB of records
```

The log stops after the first line: "import done" never appears, and there is no error message. Something stopped the process from outside.

Investigation

How did the container end?

```
kubectl get pod import -n trouble-12 -o jsonpath='{reason:
{.status.containerStatuses[0].state.terminated.reason}, exit code
{.status.containerStatuses[0].state.terminated.exitCode}}{"\n"}'
kubectl get pod import -n trouble-12 -o jsonpath='{memory limit: {.spec.containers[0].resources.limits.memory}
{"\n"}'
```

```
reason: OOMKilled, exit code 137
memory limit: 48Mi
```

`OOMKilled`, exit code 137 (128 + signal 9, SIGKILL): the kernel's out-of-memory killer stopped the process because the container used more memory than its **limit**, 48 MiB. The program had no chance to log anything.

Root cause

The import needs about 100 MiB but is limited to 48 MiB. A memory limit is a hard wall (lesson 15): crossing it kills the process, it does not slow it down.

Fix

Give the job the memory it needs (a Pod's resources cannot be changed in place, so recreate it):

```
kubectl delete pod import -n trouble-12
sed 's/memory: 48Mi/memory: 192Mi/' labs/12-troubleshooting/manifests/12-oomkilled.yaml | kubectl apply -n trouble-12 -f -
kubectl wait --for=jsonpath='{.status.phase}'=Succeeded pod/import -n trouble-12 --timeout=120s > /dev/null
kubectl logs import -n trouble-12
```

Verification

```
kubectl get pod import -n trouble-12
```

Lesson learned

- `OOMKilled` / exit code 137 / logs that just stop = the memory limit, not a bug in the logs.
- Set limits from measured usage plus headroom (`kubectl top pod` with metrics-server, lesson 26).
- If memory keeps growing until the kill, raising the limit only delays it: that is a leak to fix in the program.

Cleanup

🔪 Delete the namespace `trouble-12`:

```
kubectl delete namespace trouble-12
```

Problem 13 · DNS resolution failure

Lab 12 · Troubleshooting · ⌚ 10 minutes · run every command from the course folder · cluster: minikube

Problem

The reporting team's Pod, in its own namespace, must call the backend. "It works from the backend's namespace but not from ours." Set up the backend in `trouble-13` and a client namespace `trouble-13-client`:

```
kubectl create namespace trouble-13
kubectl create namespace trouble-13-client
kubectl apply -f labs/12-troubleshooting/manifests/13-dns.yaml
kubectl rollout status deployment/backend -n trouble-13 --timeout=120s
```

Symptoms

```
kubectl run client -n trouble-13-client --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
sh -c 'wget -qO- -T 5 http://backend 2>&1' || true
```

```
wget: bad address 'backend'
pod trouble-13-client/client terminated (Error)
```

`bad address`: the name `backend` did not resolve to any address.

First command to run

Is the cluster DNS working at all? Resolve a name that must exist, the API server's Service:

```
kubectl run dns -n trouble-13-client --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
nslookup kubernetes.default.svc.cluster.local 2>&1 | grep -A1 '^Name'
```

```
Name:   kubernetes.default.svc.cluster.local
Address: 10.96.0.1
--
Name:   kubernetes.default.svc.cluster.local
Address: 10.96.0.1
```

DNS works. So the problem is the **name**.

Investigation

A short Service name is completed with the **caller's** namespace (the `search` line of the Pod's `/etc/resolv.conf`):

```
kubectl run resolv -n trouble-13-client --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
grep search /etc/resolv.conf
```

```
search trouble-13-client.svc.cluster.local svc.cluster.local cluster.local
warning: couldn't attach to pod/resolv, falling back to streaming logs: unable to upgrade connection:
container resolv not found in pod resolv_trouble-13-client
search trouble-13-client.svc.cluster.local svc.cluster.local cluster.local
```

From `trouble-13-client`, `backend` means `backend.trouble-13-client.svc.cluster.local`, which does not exist. The backend Service lives in `trouble-13`:

```
kubectl get services -A --field-selector metadata.name=backend
```

Root cause

The client uses the short name `backend` from another namespace. Short names only work inside the Service's own namespace (lesson 10).

Fix

Use the Service's name with its namespace (`backend.trouble-13`), or the full name (`backend.trouble-13.svc.cluster.local`):

```
kubectl run client2 -n trouble-13-client --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
  wget -qO- -T 5 http://backend.trouble-13
```

Verification

```
kubectl run client3 -n trouble-13-client --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
  wget -qO- -T 5 http://backend.trouble-13.svc.cluster.local
```

Lesson learned

- `bad address` / `Name or service not known` = DNS; first prove DNS works with `kubernetes.default`, then check the name.
- `SERVICE` works in the same namespace; `SERVICE.NAMESPACE` across namespaces; the full name everywhere.
- In configuration (ConfigMaps), prefer `SERVICE.NAMESPACE` for anything another team owns.

Cleanup

✂ Delete the namespaces `trouble-13` and `trouble-13-client`:

```
kubectl delete namespace trouble-13 trouble-13-client
```

Problem 14 · NetworkPolicy blocks required traffic

Lab 12 · Troubleshooting · ⌚ 15 minutes · run every command from the course folder · cluster: minikube

Problem

Security locked the backend down: "only the frontend may call it". Now the frontend cannot reach it either. Set it up, with a client Pod labelled like the frontend (`app=frontend`):

```
kubectl create namespace trouble-14
kubectl apply -f labs/12-troubleshooting/manifests/14-networkpolicy.yaml -n trouble-14
kubectl rollout status deployment/backend -n trouble-14 --timeout=120s
```

Symptoms

```
kubectl run frontend -n trouble-14 --labels=app=frontend --rm -i --quiet --restart=Never --image=busybox:1.37
-- \
sh -c 'wget -q0- -T 5 http://backend 2>&1' || true
```

```
wget: download timed out
pod trouble-14/frontend terminated (Error)
```

A **timeout**, not `Connection refused`: the packets are dropped silently. That is the signature of a NetworkPolicy (with Calico, lesson 22), not of a missing endpoint (problem 05).

First command to run

Which policies apply to the backend Pods?

```
kubectl get networkpolicy -n trouble-14
```

NAME	POD-SELECTOR	AGE
backend-allow-frontend	app=backend	8s
backend-deny-all	app=backend	8s

Investigation

`backend-deny-all` blocks all incoming traffic; `backend-allow-frontend` should open it for the frontend. Policies add up: traffic is allowed if **any** policy allows it. Who does the allow rule select?

```
kubectl describe networkpolicy backend-allow-frontend -n trouble-14 | grep -A6 'Allowing ingress'
```

```
Allowing ingress traffic:
  To Port: 8080/TCP
  From:
    PodSelector: role=frontend
Not affecting egress traffic
Policy Types: Ingress
```

The rule allows Pods labelled `role=frontend`. The frontend is labelled `app=frontend`. The Service and endpoints are fine:

```
kubectl describe service backend -n trouble-14 | grep '^Endpoints'
```

Root cause

The allow rule selects the frontend by a label it does not have, so only `backend-deny-all` takes effect: every caller, the frontend included, is dropped.

Fix

```
kubectl patch networkpolicy backend-allow-frontend -n trouble-14 --type=json \
-p ' [{"op": "replace", "path": "/spec/ingress/0/from/0/podSelector/matchLabels", "value": {"app": "frontend"}} ]'
```

Verification

The frontend gets through; any other Pod is still blocked:

```
kubectl run frontend2 -n trouble-14 --labels=app=frontend --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
sh -c 'for i in $(seq 1 10); do wget -qO- -T 5 http://backend && exit 0; sleep 2; done; exit 1'
```

```
kubectl run stranger -n trouble-14 --rm -i --quiet --restart=Never --image=busybox:1.37 -- \
sh -c 'wget -qO- -T 5 http://backend 2>&1' || true
```

Lesson learned

- A timeout to a Service with endpoints = traffic dropped: suspect a NetworkPolicy.
- Policies are allow-lists that add up; a typo in a selector silently allows nothing.
- `kubectl describe networkpolicy` shows exactly whom a rule allows; compare it with `kubectl get pods --show-labels`.

Cleanup

✂ Delete the namespace `trouble-14` (Deployment, Service, both policies):

```
kubectl delete namespace trouble-14
```

Challenges

Each challenge gives you a task, requirements and hints; the solution is folded away under the task. Try first, look afterwards. Every solution is tested on the course cluster.

Beginner (after levels 1–5)

#	Challenge	Practises
01	A Pod in its own namespace	namespaces, Pods
02	Find Pods by their labels	labels, selectors
03	A well-formed Deployment	Deployments, requests, limits, readiness
04	Release, notice, roll back	rolling updates, rollbacks
05	Expose the backend and call it by name	Services, DNS

Intermediate (after levels 6–13)

#	Challenge	Practises
01	Configure the backend from a ConfigMap and a Secret	ConfigMaps, Secrets
02	A database that keeps its data	PVCs, storage
03	Health checks for a slow backend	startup, liveness, readiness probes
04	A guaranteed Pod on a labelled node	requests, limits, scheduling
05	A nightly report, tested now	CronJobs, Jobs
06	Two applications behind one host name	Ingress
07	Only the API may reach the cache	NetworkPolicies
08	A read-only identity for a dashboard	ServiceAccounts, RBAC

Troubleshooting (after level 15)

#	Challenge	The symptom
01	The Service has endpoints, but refuses	connection refused
02	Restarts every few seconds	restart count climbing
03	The container never starts	RunContainerError
04	The export Job gives up	BackoffLimitExceeded
05	Only half of the replicas	a Deployment short of Pods
06	Pending on an empty cluster	FailedScheduling

The troubleshooting challenges' own [README](#) explains how to start each broken setup.

Glossary

Every Kubernetes term the course uses, in alphabetical order, with the lesson that explains it.

Term	Meaning	Lesson
Annotation	a key/value note on an object for tools and people; unlike labels, never used to select objects	07
API server	the front door of the cluster: every command and every component talks only to it	02
ClusterIP	the default Service type: a stable virtual IP reachable only inside the cluster	10
ClusterRole / ClusterRoleBinding	RBAC permissions (and their assignment) that apply to the whole cluster or to cluster-wide objects	24
ConfigMap	non-secret configuration stored in the cluster, given to Pods as environment variables or files	11
Container runtime	the program on each node that actually runs containers (containerd in Minikube)	02
Context	a named kubeconfig entry: cluster + user + default namespace; <code>kubectl config current-context</code>	04
Controller manager	runs the reconciliation loops that keep reality equal to the desired state	02
CoreDNS	the cluster's DNS server: resolves Service names such as <code>backend</code> or <code>backend.ns.svc.cluster.local</code>	10
CronJob	creates a Job on a schedule (cron syntax)	17
DaemonSet	runs one Pod on every (suitable) node: log and monitoring agents	18
Deployment	manages ReplicaSets to keep N identical Pods running, with rolling updates and rollbacks	09
Desired state	what you declared in YAML; Kubernetes continuously makes reality match it (reconciliation)	01
emptyDir	a volume that lives as long as its Pod: scratch space shared by the Pod's containers	13
Endpoints / EndpointSlice	the list of Pod addresses behind a Service; empty when the selector matches no ready Pod	10
etcd	the key-value database that stores every object of the cluster	02
Event	a record of something that happened to an object (scheduled, pulled, failed...); <code>kubectl get events</code>	27
Headless Service	a Service with <code>clusterIP: None</code> : DNS returns the Pods' own addresses (StatefulSets)	19
Helm / chart / release	the package manager for Kubernetes; a chart is a package of templates, a release one installation of it	28
HorizontalPodAutoscaler (HPA)	changes a Deployment's replica count from metrics such as CPU	26
Ingress / ingress controller	HTTP routing rules (hosts, paths) to Services, applied by a controller such as ingress-nginx	20
Job	runs Pods until a task completes successfully	17
kube-proxy	makes Service virtual IPs work on every node	02
kubeconfig	the file (<code>~/.kube/config</code>) that tells kubectl which clusters exist and how to log in	03

Term	Meaning	Lesson
kubectl	the command-line client of the Kubernetes API	04
kubelet	the agent on every node: starts a Pod's containers, runs its probes, reports its status	02
Label / selector	key/value tags on objects, and the queries that pick objects by them (Services, Deployments, policies)	07
Limit / request	the most a container may use, and what the scheduler reserves for it (CPU, memory)	15
Liveness / readiness / startup probe	restart if dead / send traffic only when ready / wait for a slow start	14
LoadBalancer	a Service type that asks the infrastructure (cloud) for an external load balancer	10
Minikube	a local one-node Kubernetes cluster, running in Docker for this course	03
Namespace	a named group of objects inside a cluster, for organisation, access control and quotas	05
NetworkPolicy	firewall rules between Pods by label and namespace (enforced by the network plugin, Calico here)	22
Node	a machine (VM or server) that runs Pods	02
nodeSelector	runs a Pod only on nodes with the given labels	16
NodePort	a Service type that also opens a port (30000–32767) on every node	10
OOMKilled	the container exceeded its memory limit and the kernel killed it (exit code 137)	15
PersistentVolume (PV)	a piece of storage in the cluster, created by an administrator or a provisioner	13
PersistentVolumeClaim (PVC)	a Pod's request for storage; bound to a PV	13
Pod	the smallest deployable unit: one or more containers sharing network and storage	06
RBAC	role-based access control: Roles (what) bound to users or ServiceAccounts (who)	24
Reconciliation	the control loop "observe → compare with desired state → act", the core of Kubernetes	01
ReplicaSet	keeps N identical Pods running; normally managed by a Deployment	08
Role / RoleBinding	permissions inside one namespace, and their assignment to a user, group or ServiceAccount	24
Rolling update / rollback	replacing Pods gradually with a new version / returning to a previous revision	09
Scheduler	picks a node for each new Pod, based on resources, selectors, taints and more	02
Secret	sensitive configuration (passwords, tokens); base64-encoded, not encrypted by default	12
Service	a stable name and virtual IP in front of a changing set of Pods	10
ServiceAccount	the identity of a workload (Pod) when it talks to the Kubernetes API	23
Sidecar	a helper container in the same Pod as the application (log shipper, proxy)	06
StatefulSet	like a Deployment, but with stable Pod names, ordered start and one volume per Pod	19
StorageClass	a kind of storage and the provisioner that creates PVs for PVCs on demand	13
Taint / toleration	a node repels Pods (taint) unless a Pod accepts it (toleration)	16
Static Pod	a Pod the kubelet runs from a file on its node; how the control plane starts itself	02

Kubernetes interview questions

Questions asked in junior and mid-level DevOps and platform interviews. Answer out loud first, then open the model answer; the lesson link has the hands-on version.

1. What problem does Kubernetes solve that Docker alone does not?

▼ Answer

Docker runs containers on one machine. Kubernetes manages containerized applications across many machines from a declared desired state: it keeps the right number of copies running (self-healing), places them on nodes, gives them stable addresses (Services), spreads traffic, scales them and updates them without downtime. → [01](#)

2. Name the control-plane components and what each does.

▼ Answer

API server (the only entry point), etcd (the state store), scheduler (picks a node for new Pods), controller manager (reconciliation loops). On every node: kubelet (runs the Pods), kube-proxy (Service networking), container runtime. → [02](#)

3. A Pod stays `Pending`. How do you find out why?

▼ Answer

`kubectl describe pod` and read the Events: `FailedScheduling` with `Insufficient cpu/memory`, an untolerated taint, a nodeSelector no node matches, or a PVC that is not bound. No events at all means nothing is scheduling (the scheduler is down). → [27](#), [02](#)

4. What is the difference between a Pod, a ReplicaSet and a Deployment?

▼ Answer

A Pod runs containers. A ReplicaSet keeps N identical Pods running. A Deployment manages ReplicaSets so it can roll out a new version (a new ReplicaSet) gradually and roll back to an old one. You normally create Deployments only. → [08](#), [09](#)

5. How does a Service find its Pods, and what happens when a Pod is replaced?

▼ Answer

By its label selector: every ready Pod whose labels match becomes an endpoint. A replaced Pod gets a new IP; the Service's endpoints update automatically and the Service's name and IP stay the same. → [10](#)

6. ClusterIP, NodePort, LoadBalancer, Ingress: when do you use which?

▼ Answer

ClusterIP for traffic inside the cluster (the default). NodePort to open a port on every node (testing, simple setups). LoadBalancer for an external load balancer from the cloud, one per Service. Ingress for HTTP(S) routing of many hosts/paths to many Services behind one entry point. → [10](#), [20](#)

7. A Service has no endpoints. What are the usual causes?

▼ Answer

The selector does not match the Pods' labels (typo), the Pods are not ready (failing readiness probe), or they are in another namespace. `kubectl get endpointslices`, `kubectl get pods --show-labels`, `kubectl describe pod`. → [10](#), [labs/12](#)

8. ConfigMap vs Secret: what is the difference, and is a Secret encrypted?

▼ Answer

Both hold configuration for Pods; Secrets are for sensitive values and are handled more carefully (RBAC, not shown by default). By default a Secret is only base64-encoded, not encrypted: anyone who can read it, or read etcd, sees the value. Encryption at rest, RBAC and external secret managers are needed for real protection. → [12](#)

9. Liveness, readiness and startup probes: what does each one do when it fails?

▼ Answer

Liveness: the kubelet restarts the container. Readiness: the Pod is removed from its Services' endpoints (no traffic), not restarted. Startup: liveness and readiness wait until it succeeds, for slow-starting applications. → [14](#)

10. What are requests and limits, and what happens when a container exceeds them?

▼ Answer

Requests are what the scheduler reserves for the container; limits are the maximum. Above the CPU limit the container is throttled; above the memory limit it is OOMKilled (exit code 137) and restarted. → [15](#)

11. How does a rolling update work, and how do you roll back?

▼ Answer

The Deployment creates a new ReplicaSet and shifts Pods from the old one to the new one within `maxSurge` and `maxUnavailable`, waiting for new Pods to be ready. `kubectl rollout status` follows it, `kubectl rollout undo` returns to the previous revision. → [09](#)

12. A rollout is stuck. What do you check?

▼ Answer

`kubectl rollout status` and the new Pods: `ImagePullBackOff` (wrong image or tag), `CrashLoopBackOff`, or a readiness probe that never passes. The events and logs of the new Pod name the cause; `rollout undo` restores service. → [09](#), [29](#)

13. PV, PVC and StorageClass: how do they relate?

▼ Answer

A Pod uses a PVC (a request for storage); the PVC binds to a PV (actual storage). With a StorageClass, a provisioner creates the PV automatically when the PVC is created (dynamic provisioning). → [13](#)

14. Deployment vs StatefulSet?

▼ Answer

Deployment Pods are interchangeable (random names, shared or no storage). StatefulSet Pods have stable names (`db-0`), start in order, keep their own PVC each and get stable DNS through a headless Service: databases and clustered systems. → 19

15. Job, CronJob, DaemonSet: give one use for each.

▼ Answer

Job: a database migration that must complete once. CronJob: a nightly backup. DaemonSet: a log or monitoring agent on every node. → 17, 18

16. How does service discovery work in Kubernetes?

▼ Answer

CoreDNS resolves `SERVICE` (same namespace), `SERVICE.NAMESPACE` and `SERVICE.NAMESPACE.svc.cluster.local` to the Service's ClusterIP; Pods use the name, never Pod IPs. → 10, 21

17. What does a NetworkPolicy do, and what is needed for it to work?

▼ Answer

It restricts which Pods (by label and namespace) may talk to which, on which ports. Once a Pod is selected by a policy, everything not allowed is denied. It needs a network plugin that enforces policies (Calico, Cilium); otherwise it is silently ignored. → 22

18. How would you give a developer read-only access to one namespace?

▼ Answer

A Role with `get`, `list`, `watch` on the resources they need, and a RoleBinding to their user or group in that namespace; verify with `kubectl auth can-i delete pods -n NS --as USER` → `no`. → 24

19. What is a ServiceAccount, and why disable its token when not needed?

▼ Answer

The identity of a Pod towards the Kubernetes API. Every Pod gets one; if the application never calls the API, setting `automountServiceAccountToken: false` removes a credential an attacker could steal. → 23

20. How does the Horizontal Pod Autoscaler decide how many Pods to run?

▼ Answer

It reads metrics (CPU from metrics-server, relative to the Pods' requests) and computes `desired = ceil(current × currentMetric / target)`, within min and max replicas, with a stabilization window before scaling down. Without requests or metrics it cannot work. → 26

21. Why use Helm?

▼ Answer

To package an application's manifests as one versioned chart with values per environment, and to install, upgrade and roll back the whole set as a release instead of applying many YAML files by hand. → 28

22. Walk me through troubleshooting "the application does not respond".

▼ Answer

From the outside in: Ingress (rules, controller) → Service (endpoints, selector, ports) → Pods (status, restarts, readiness) → events → logs (`--previous` after a crash) → configuration (ConfigMaps, Secrets) → dependencies (DNS, database) → NetworkPolicies. Fix the source file, apply it, verify as the user. → [27](#), [29](#)